

Optimal gait design for a soft quadruped robot via multi-fidelity Bayesian optimization

Kaige Tan, Xuezhi Niu, Qinglei Ji, Lei Feng*, Martin Törngren

Department of Engineering Design, KTH Royal Institute of Technology, Stockholm 10044, Sweden

ARTICLE INFO

Keywords:

Soft quadruped robot
Reality gap
Multi-fidelity Bayesian optimization
Edge computing

ABSTRACT

This study focuses on the locomotion capability improvement in a tendon-driven soft quadruped robot through an online adaptive learning approach. Leveraging the inverse kinematics model of the soft quadruped robot, we employ a central pattern generator to design a parametric gait pattern, and use Bayesian optimization (BO) to find the optimal parameters. Further, to address the challenges of modeling discrepancies, we implement a multi-fidelity BO approach, combining data from both simulation and physical experiments throughout training and optimization. This strategy enables the adaptive refinement of the gait pattern and ensures a smooth transition from simulation to real-world deployment for the controller. Compared to previous result using a fixed gait pattern, the multi-fidelity BO approach improves the robot's average walking speed from 0.14 m/s to 0.214 m/s, an increase of 52.7%. Moreover, we integrate a computational task off-loading architecture by edge computing, which reduces the onboard computational and memory overhead, to improve real-time control performance and facilitate an effective online learning process. The proposed approach successfully achieves optimal walking gait design for physical deployment with high efficiency, effectively addressing challenges related to the reality gap in soft robotics.

1. Introduction

Soft robotics, which employs compliant materials and structures as actuators, is increasingly being adopted for building adaptive and resilient robotic systems. This approach is especially prominent in the design of soft quadruped robots, which are used for legged locomotion and are gaining attention for their ability to emulate the flexibility of biological tissues, unlike their rigid counterparts [1]. These robots use a variety of actuation mechanisms, such as pneumatic pressure [2,3], electric voltage [4,5], and magnetic fields [6,7]. Such actuators enable soft robots to demonstrate resilience under various working conditions, enhance safety in human-machine interactions, and effectively absorb external disturbances due to their elastic properties [8].

While soft actuators facilitate intricate deformations for complex locomotion, effective gait design is the necessity of exploiting the full walking potential of soft robots [9]. The challenges arise from the soft robots' modeling complexity under environmental interactions and maintaining consistent actuation forces with precision [10].

To address these challenges, tendon-driven soft actuators (TSAs) are employed, synchronizing bending and compression motions to improve walking performance [11]. Prior research has predominantly focused on predefined gait trajectories [9] or model-based control designs for gait optimization [11]. However, these approaches either neglect gait

trajectory optimization or rely solely on Finite Element Analysis [12, 13]. This highlights a research gap in advanced gait exploration for TSA-type soft robots to develop optimal gait patterns.

To create adaptive and efficient locomotion patterns, many state-of-the-art techniques provide sophisticated solutions tailored for rigid quadruped robots [14]. When applying these techniques to soft robots, two major challenges arise:

- **Uncertain parameter optimization:** Owing to the intrinsic non-linear and time-variant properties of the soft material, the dynamics of the soft actuators is subject to significant uncertainty. Traditional deterministic functions and optimization methods lack robustness when applied to the soft robot gait design.
- **Modeling error compensation:** Precisely modeling the dynamics of soft materials is challenging due to their compliant nature. Consequently, it is imperative to address the discrepancies between the simulation and the real-world performance of soft robots. The inherent complexities of soft robotic systems pose challenges for simulators to accurately replicate them. Therefore, it becomes crucial to devise strategies for reducing this modeling error during physical deployment.

* Corresponding author.

E-mail address: lfeng@kth.se (L. Feng).

<https://doi.org/10.1016/j.asoc.2024.112568>

Received 29 May 2024; Received in revised form 28 October 2024; Accepted 23 November 2024

Available online 3 December 2024

1568-4946/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

To address the first challenge, the implicit relationship between the model parameters and the objective function is generally treated as a stochastic black-box model, where optimal parameter values can be determined through black-box optimization methodologies. Solving black-box optimization via soft computing has proven effective in various robotics applications. Notably, algorithms such as Bayesian Optimization (BO) [15], Particle Swarm Optimization (PSO) [16], and Evolutionary Algorithms (EA) [17] have been extensively employed. Among them, BO has the advantage of considering probabilistic objective functions and using efficient sampling to achieve fast convergence. Thus, given the parametric-based gait pattern, BO stands out as an ideal choice for systematically tuning uncertain parameters, thereby unlocking the potential of the robot's gait with high exploration efficiency.

The second challenge stems from the inevitable discrepancies between the simulation model and the real robot, owing to noise, perturbations, and modeling errors. This disparity, often referred to as the reality gap, leads to discrepancies within a simulator-aided framework, necessitating adjustments to the optimal solution identified in the simulation environment through physical experiments [18]. When developing a data-driven controller, abundant synthetic training data can be obtained from the simulator, but the data contain unavoidable errors compared to data derived from real robots. Moreover, while measurement data from real robots are of high quality, obtaining them in large quantities is challenging due to the need for human supervision and experiment resets, resulting in a slow and costly process [19]. To address this challenge, in this study, we propose a multi-fidelity BO (MFBO) method that mixes the two types of data in a single learning algorithm to alleviate the reality gap between simulation and physical robots. Consequently, the optimal walking pattern learned from simulations can be efficiently adapted to the physical robot through online learning in the real world.

Moreover, the complexities of online MFBO training and the challenges associated with Central Pattern Generator (CPG) gait generation pose difficulties in achieving real-time control with the robot's onboard computation system. Recently, the integration of edge computing has emerged as a promising solution to enhance robotic system performance [20]. Computational task off-loading becomes instrumental in load balancing [21] and optimizing resource utilization [22]. Furthermore, compared to task off-loading to the public cloud, the integration of edge computing allows for data processing closer to the source with less latency, ensures real-time computational task completion [10], and improves the system responsiveness [23]. These technological advancements unlock the full potential of learning-enhanced control systems, providing high-performance computing capabilities and alleviating computation complexity challenges associated with real-time constraints. Owing to the high complexity of the training process, this study leverages the computational off-loading enabled by edge computing to improve the real-time control performance for the soft quadruped robot gait design. Situated at the intersection of these technological frontiers, we present an exploration of the synergies between online optimization-based control and computational task off-loading in the context of soft quadruped robots.

To sum up, this paper investigates the following research questions (RQs).

- Since there is neither explicit nor deterministic function between the gait parameters and the walking quality of the soft robot, how can we efficiently find the optimal or close-to-optimal parameters for high-quality walking of the soft robot?
- Since the reality gap between simulation models and physical robots are unavoidable, how can we efficiently adapt a controller learned from simulation models to the physical robots so that the control performance on the physical robots is similar to that on the models?

Table 1

List of abbreviations.

Abbreviations	Definition
TSA	tendon-driven soft actuator
CPG	Central Pattern Generator
BO	Bayesian Optimization
PSO	Particle Swarm Optimization
EA	Evolutionary Algorithm
MFBO	multi-fidelity Bayesian Optimization
GP	Gaussian Process
MTGP	multi-task Gaussian Process
RF	random forest
PL	piece-wise linear
EI	expected improvement
PI	probability of improvement
GP-UCB	Gaussian Process upper confidence bounds
IMU	inertial measurement unit
ToF	Time of Flight
PD	proportional-derivative
IoT	Internet of Things
AGS	Adaptive grid search
SA	Simulated annealing
GA	Genetic Algorithms
RS	Random search
OWD	Omni-directional walking design
RFE	Random Fourier Expansion
SVM	Support Vector Machine
NMSE	normalized mean squared error

- When the training data set is large, the learning algorithm may be too time consuming to be completed in real-time constraints. Can task offloading to an edge server complete the training algorithms without compromising quality and within the real-time constraints for the optimal gait control of the soft robot?

The answers to these research questions form a comprehensive solution that addresses the key challenges in soft robot gait design and optimization. The improvements in real-time adaptability, scalability, and computational efficiency make this approach a strong candidate for various sectors requiring robust and adaptive robotic systems. These benefits directly address industry needs for flexible, resilient, and cost-effective robotic solutions capable of performing in diverse operational conditions. In summary, the contributions of our article are as follows.

1. A gait generator is devised for the soft quadruped robot with CPG-based and Hopf oscillator modeling. The utilization of a parametric formulation enhances the adaptability and flexibility of gait pattern design.
2. The computation of the optimal gait pattern parameters is formulated as a stochastic black-box optimization problem, which is solved using the proposed MFBO approach. Our approach effectively mitigates the reality gap and achieves a rapid transition from sim-to-real adaptation.
3. An online learning architecture is designed based on an edge computing framework. Complex MFBO optimization and training tasks are performed at the edge server. Computational task off-loading, facilitated by 5G communication, allows for real-time adaptive parameter optimization and fast deployment.

The list of abbreviations used in this paper is summarized in Table 1. The rest of this article is organized as follows. Section 2 reviews related works on gait locomotion control and relevant technologies applied in this work. Followed by the preliminaries in Section 3, Section 4 introduces the soft quadruped robot under study and CPG-based gait pattern model design. Then, the formulated parameter optimization problem is solved by MFBO in Section 5. Section 6 describes the hardware implementation and the experiment setup. Section 7 shows the evaluation and results of the approach, and Section 8 summarizes and discusses the results and findings. Finally, Section 9 concludes this article.

2. Related works

2.1. Gait locomotion design

To create adaptive and efficient locomotion patterns, many state-of-the-art techniques provide sophisticated solutions tailored for rigid quadruped robots. Knowledge-based gait design relies on explicit mathematical models of a robot's dynamics to create efficient, predictable locomotion strategies. Common approaches include trajectory optimization, where the robot's gait is optimized based on energy or torque minimization while satisfying physical constraints. Model Predictive Control (MPC) [24,25] is often used to adjust gaits in real time, ensuring stable and efficient movement. In addition, Central Pattern Generators (CPGs), inspired by biological systems, use oscillator networks [15] to produce rhythmic signals for smooth, stable gait patterns. Variants like Hopf oscillators [26] and van-der-Pol oscillators [27] can generate rhythmic signals in the form of trajectories and represent the stance-swing sequence of the legs with continuous states, making them well-suited for modeling gait patterns in soft quadruped robots. Moreover, inverse kinematics and dynamics help compute the necessary joint configurations and torques for precise control. Our previous work [8] proposed an Omni-directional walking design (OWD), that generates smooth and continuous gait patterns enabling a quadruped robot to move in any direction. It employs predefined gait parameters and yields satisfactory performance in controlled environments. However, the fixed parameters have limited adaptability in dynamic or unpredictable settings. Therefore, future research is needed to address real-time adaptability and responsiveness to environmental changes.

On the other hand, data-driven approaches do not rely on explicit models of the system dynamics. Instead, they treat the locomotion design problem as a black box, where the target is to optimize the gait performance based solely on evaluations of a cost or reward function. This allows for the exploration of complex, high-dimensional control action spaces that are difficult to model analytically. Reinforcement Learning (RL) approaches are the dominant in this category, and they focus on directly generating the optimal state-action pair as a policy for gait control. They model the problem as a Markov Decision Process (MDP), where the robot interacts with its environment to maximize cumulative rewards. By continuously improving the policy through trial and error, RL techniques like soft Actor-Critic (SAC) [19,28] and Proximal Policy Optimization (PPO) [29] learn efficient gaits that adapt to dynamic environments. They are advantageous when the goal is to develop adaptive behaviors without relying on predefined models.

In contrast, hybrid methods combine model-based techniques with machine learning to reduce computational complexity while maintaining performance. They can both provide efficiency when accurate models are difficult to obtain and offer stability in control compared to pure data-driven methods. Typical approaches in this category are black-box optimization approaches, where they do not explicitly generate a policy but rather focus on parameterizing a model. Kim et al. [30] perform gait optimization of a quadruped robot using foot placement perturbation based on an evolutionary computation approach. Widner et al. [31] apply Bayesian optimization based on predefined gait parameterization to learn optimal gait parameters thanks to its sample efficiency. In addition, particle swarm optimization is employed to develop different types of gaits based on predefined motion patterns [32]. These methods rely on iterative evaluations of a cost or reward function to optimize the parameters, and they are more general and can handle a wider variety of control architectures.

2.2. Reality gap mitigation for robot applications

In robotics, the reality gap refers to the discrepancies between simulated and real-world environments, which poses significant challenges when transferring control policies or models developed in simulations to physical robots [18]. To address this issue, several methods have

been proposed to mitigate the reality gap, ensuring that controllers or inference algorithms developed in simulations perform robustly in real environments. One common approach to reality gap mitigation is domain randomization, where variations in the simulation environment (e.g., changes in friction, mass, or sensor noise) are introduced to expose the controller to a wide range of conditions [33], effectively preparing it for the unpredictability of the real world. Horváth et al. [34] propose a domain randomization method for object detection that can differentiate similar classes of objects using a few real examples. Muratore et al. [35] propose a Bayesian domain randomization approach to solve sim-to-real issues by adapting the parameter distribution during learning given sparse data from the real-world target domain.

Another key technique is transfer learning, where controllers initially trained in simulations are fine-tuned on the real robot using real-world data. This method involves pre-training the model or policy in simulation and then refining it based on feedback from the physical system. This approach can drastically reduce the time and resources needed for real-world testing by leveraging the efficiency of simulations. Jaquier et al. [36] provide a comprehensive review of transfer learning techniques for robotics, highlighting how policies can adapt from simulation to real-world scenarios. A residual learning approach [37] is proposed, where skill acquisition occurs in simulation during the offline phase, and admittance learning is performed on the real robot during online execution for safe and stable interactions with the real-world environment.

To further enhance the transfer learning process, MFBO has emerged as a promising approach. MFBO extends the traditional BO framework to incorporate information from diverse sources with different fidelity levels. While BO typically relies on a Gaussian Process (GP) model as a surrogate to represent objective function evaluations, it is limited by a scalar output for a single model characterization. To address this limitation, either separate GPs are used for distinct models [38], or an error function is trained to delineate the difference among different models [39]. Additionally, the multi-task GP (MTGP) method [40,41] has been introduced to naturally manage multiple coupled outputs, capturing complex data dependencies arising from various sources. Incorporating MTGP into the BO framework gives rise to the MFBO method, which facilitates accelerated model training across different data sources. Recently, MFBO has found applications in various research domains, e.g., hyper-parameter optimization [42], process optimization [43], and safe controller design [44]. However, its potential for modeling error compensation in robotics systems, particularly in adaptive controller design, remains underexplored.

Similar to our paper, Lan et al. [15] integrate BO and CPG for the robot's optimal gait pattern generation. Although they identify the reality gap during the physical testing, they do not propose a method to adapt the gait pattern to the physical robot. The shortcoming prompts our exploration of the MFBO approach to the efficient adaptation of the gait controller learned from simulations to the physical robot. The adaptation procedure is performed through online learning with the real robot, which combines both previously obtained simulation data and the online generated real-world data to obtain a better gait controller for the physical robot. Note that prior studies do not distinguish the terms multi-fidelity and multi-task within the same BO framework. We adopt the term multi-fidelity in this paper to maintain clarity and underscore its relevance within the context of sim-to-real transfer scenarios [45].

2.3. Edge computing for robotics

As robotic systems become increasingly complex and data-intensive, the need for real-time processing and decision-making is paramount. Edge computing has emerged as a powerful solution to meet these demands by bringing computational resources closer to the data source, such as sensors and actuators on the robot itself [46]. In contrast to

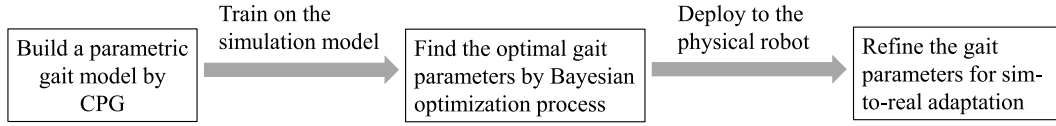


Fig. 1. Optimal gait control policy generation framework by multi-fidelity Bayesian optimization.

cloud computing, where data is sent to a central server for processing, edge computing enables robots to process data locally or in nearby edge devices, reducing latency and improving system responsiveness [20].

In robotic applications, edge computing has been applied in various scenarios and offers several advantages. First, it allows for real-time data analysis and control, which is critical for tasks that require immediate responses, such as autonomous navigation [47], object manipulation [48], and human–robot interaction [49]. By processing sensor data at the edge, robots can make decisions faster without relying on cloud-based systems that may introduce delays due to network latency [46]. This is particularly important in environments where high-speed internet access is unavailable or unreliable. Second, edge computing enhances data privacy and security. Since sensitive data, such as visual feeds or personal information from users, can be processed locally on the robot, the risk of exposing this data to external networks is reduced [50]. This is especially beneficial in applications like industrial robotics [51], where privacy is a critical concern. Additionally, edge computing supports scalability and efficient resource utilization. For instance, edge computing facilitates collaborative robot systems, where multiple robots share computational resources, enhancing collective performance in scenarios like swarm robotics [52] or distributed sensor networks [53].

3. Preliminaries

3.1. Method overview

This study aims to develop a gait locomotion control strategy for soft quadruped robots with high efficiency and real-time adaptability. An overview of our methodology is given in Fig. 1. The method for optimizing the gait of the soft quadruped robot integrates a CPG, BO, and MFBO to achieve adaptive and efficient locomotion. Using Hopf oscillators [26], the CPG generates a parametric controller with rhythmic signals to control the robot's leg movements, where its parameters such as amplitude and phase are crucial for synchronizing gait patterns. The inclusion of Hopf oscillators as part of the CPG framework is crucial, as these oscillators generate stable and synchronized trajectories for the robot's leg movements. BO is then employed to optimize these parameters by efficiently exploring the design space, where a GP regression model is applied as a surrogate model to predict the optimal gait pattern. To address discrepancies between simulation and real-world performance, MFBO incorporates data from both simulation and physical experiments, ensuring a smooth transfer of the optimized parameters to the physical robot. Additionally, the computationally intensive tasks of BO and MFBO are offloaded to an edge server, allowing for real-time gait adjustments and maintaining low onboard computational demands.

3.2. Gaussian processes model

A GP model represents a stochastic function $g : \mathcal{P} \rightarrow \mathbb{R}$, where $\mathcal{P}$ is a subset of $\mathbb{R}^n$. A GP model is specified by its mean and covariance functions, i.e., $m : \mathcal{P} \rightarrow \mathbb{R}$ and $k : \mathcal{P} \times \mathcal{P} \rightarrow \mathbb{R}$. For a point $\mathbf{p} \in \mathcal{P}$, $g(\mathbf{p}) \sim \mathcal{GP}(m(\mathbf{p}), k(\mathbf{p}, \mathbf{p}))$ is a random value with the Gaussian distribution whose mean is $m(\mathbf{p}) = \mathbb{E}[g(\mathbf{p})] \in \mathbb{R}$ and variance is $k(\mathbf{p}, \mathbf{p})$. For any two points $\mathbf{p}, \mathbf{p}' \in \mathcal{P}$, the covariance is defined by $k(\mathbf{p}, \mathbf{p}') = \mathbb{E}[(g(\mathbf{p}) - m(\mathbf{p}))(g(\mathbf{p}') - m(\mathbf{p}'))] \in \mathbb{R}_+$. The covariance function defines the relationships between data points in the input space, which

is characterized by the hyperparameters λ . They impact the shape of regression functions and can be optimally defined by log marginal likelihood. With a training data set $\mathcal{T} = \{\mathbf{p}_i, v_i\}, i = 1, \dots, n$ of n training samples $\mathbf{P} = [\mathbf{p}_1, \dots, \mathbf{p}_n]$ and their corresponding outputs $\mathbf{v} = [v_1, \dots, v_n]$, the optimal values λ^* can be acquired by maximizing the likelihood of the observed outputs. Based on that, the GP model can be used to predict the function value v' of a test point $\mathbf{p}'$, yielding

$$p(v' | \mathbf{p}', \lambda^*, \mathcal{T}) = \mathcal{N}(\mu_{\mathbf{p}'}, \sigma_{\mathbf{p}'}), \quad (1)$$

and

$$\begin{aligned} \mu_{\mathbf{p}'} &= k(\mathbf{P}, \mathbf{p}')^\top K_{\mathbf{P}}^{-1} \mathbf{v}^\top, \\ \sigma_{\mathbf{p}'} &= k(\mathbf{p}', \mathbf{p}') - k(\mathbf{P}, \mathbf{p}')^\top K_{\mathbf{P}}^{-1} k(\mathbf{P}, \mathbf{p}'), \end{aligned} \quad (2)$$

where $\mu_{\mathbf{p}'}$ and $\sigma_{\mathbf{p}'}$ are the mean and variance of v' . $K_{\mathbf{P}}$ is the symmetric and positive semi-definite $n \times n$ covariance matrix and $K_{\mathbf{P}}(i, j) = k(\mathbf{p}_i, \mathbf{p}_j)$. $k(\mathbf{P}, \mathbf{p}')$ is an $n \times 1$ vector, and each entry denotes the kernel function value between $\mathbf{p}_i \in \mathbf{P}$ and $\mathbf{p}'$. The values of $k(\cdot, \cdot)$ and $K_{\mathbf{P}}$ in (2) depend on λ^* .

3.3. Bayesian optimization

Bayesian optimization (BO) is an effective optimization technique for black-box optimization. The method employs a data fitting surrogate model $g(\cdot)$ to approximate the unknown objective function $J(\cdot)$. The selection of the surrogate model is crucial for accuracy and efficiency [54]. Popular surrogate models include GP, random forest (RF), piece-wise linear (PL), etc. The GP model is often preferred for two primary reasons: (1) its ability to quantify uncertainty in predictions, and (2) its adaptation for mitigation of the reality gap in multi-fidelity BO. Comparisons of different surrogate models for BO are presented later in Section 7.3.

Based on the training data set and the GP model hyperparameters, Bayesian optimization maximizes an acquisition function $a(\cdot)$ to determine the next most promising point to evaluate in the search for global optimality of $J(\cdot)$, i.e., $\mathbf{p}_{\text{next}} = \text{argmax}_{\mathbf{p}} a(\mathbf{p} | \mathcal{T}, \lambda)$. After obtaining $\mathbf{p}_{\text{next}}$ and $J(\mathbf{p}_{\text{next}})$, the GP model will also be updated based on the new observation. The iteration continues until predetermined stopping conditions are met, such as a predetermined number of iterations or convergence criteria. If the stopping conditions are proper, the iteration converges to the optimal solution.

Many types of acquisition function $a(\cdot)$ have been proposed, e.g., expected improvement criterion (EI), Probability of Improvement (PI), and GP upper confidence bounds (GP-UCB). EI has proved its superiority over PI, and it does not require the tuning parameter as in GP-UCB [55]. Thus, EI is adopted in this study, which maximizes the expected improvement over the current best J^* , i.e.,

$$a_{\text{EI}}(\mathbf{p} | \mathcal{T}, \lambda) = \mathbb{E}(\max[0, g(\mathbf{p}) - J^*] | \mathcal{T}, \lambda). \quad (3)$$

It has a closed form under the GP surrogate model, i.e.,

$$\begin{aligned} a_{\text{EI}}(\mathbf{p} | \mathcal{T}, \lambda) &= \sqrt{\sigma_{\mathbf{p}}} [\gamma(\mathbf{p}) \Phi(\gamma(\mathbf{p})) + \phi(\mathbf{p})], \\ \gamma(\mathbf{p}) &= (\mu_{\mathbf{p}} - J^*) / \sqrt{\sigma_{\mathbf{p}}}, \end{aligned} \quad (4)$$

where $\mu_{\mathbf{p}}$ and $\sigma_{\mathbf{p}}$ are obtained from (2). $\phi(\cdot)$ denotes the standard normal density function, and $\Phi(\cdot)$ denotes the cumulative distribution function of the standard normal distribution.

3.4. Hopf oscillator

As a method to model the artificial CPG, the Hopf oscillator produces various harmonic output patterns and has a clear correlation with the applied coefficients. It can be denoted by the oscillator state $\mathbf{o}$, which takes the form of the nonlinear differential equations:

$$\dot{\mathbf{o}} = f_h(\mathbf{o}) + \mathbf{q} = \begin{bmatrix} k(A^2 - o_1^2 - o_2^2)o_1 - 2\pi f o_2 \\ k(A^2 - o_1^2 - o_2^2)o_2 + 2\pi f o_1 \end{bmatrix} + \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}, \quad (5)$$

where $\mathbf{o} = [o_1, o_2]^\top$ is the Hopf oscillator state vector. $\mathbf{q} = [q_1, q_2]^\top$ is a design coupling vector to coordinate different oscillators. The formulas for q_1 and q_2 are defined later in (12). A , f , and k are design parameters that determine the amplitude of the steady-state oscillation, oscillation frequency, and the speed of convergence, respectively. Given the design parameters and a non-zero initial state, a harmonic pattern will be generated from the oscillator represented by o_1 and o_2 . Note that $\mathbf{o}$ is time-dependent; however, for the sake of simplicity, we shall include the argument of time in the expression only when essential.

In addition, a dynamic modulation of frequency is introduced to generalize the oscillator formulation. It is realized by employing a varying frequency influenced by the state of oscillator, thus achieving a non-harmonic pattern. Given the shape ratio α and time constant τ , the constant frequency f in (5) can be adjusted to yield a frequency f_a , where

$$f_a(o_2) = \frac{f}{2\alpha} + \frac{(2\alpha - 1)f}{2\alpha(1 - \alpha)(1 + e^{-\tau o_2})}. \quad (6)$$

Note that the period T of the oscillator in (6) is still determined by f (i.e., $T = 1/f$), and the harmonic pattern is a special case of (6) when $\alpha = 0.5$. When $\alpha \neq 0.5$, the frequency f_a changes monotonically in each oscillation period. We direct the interested reader to Ref. [56] for the comprehensive analysis on Hopf oscillators. Note that f_a is a state-dependent variable of o_2 . As a result, the other state, i.e., o_1 , becomes the exclusive determinant of the fundamental gait trajectory, and its value serves as the output of the Hopf oscillator in the sequel.

4. Parametric gait pattern model

4.1. Inverse kinematics: Task level to motor level

Our previous works [5,8] introduce the design and fabrication of a soft quadruped robot characterized by its innovative utilization of four TSAs. The inverse kinematics model used in this study is built upon methods presented in those works. We present the essential results here to provide the necessary background and notations. Fig. 2(a) provides a visual representation of the robot, illustrating key states of a robot such as roll (θ_x), pitch (θ_y), yaw (θ_z), translational velocities along three axes (v_x, v_y, v_z), and the normal contact forces on each foot ($f_{nFL}, f_{nFR}, f_{nRL}, f_{nRR}$). The defining characteristic of the soft legs is the cable-driven mechanism, which enables seamless and continuous bending and twisting along its entire length. The cables, also referred to as tendons, guide the deformation of soft materials, facilitating precise movement to designated positions. Each leg of the robot is actuated by three cables, which are driven by servo motors, resulting in the generation of tendon displacement denoted as $\mathbf{x}_{ten} = [x_A, x_B, x_C]^\top$, as illustrated in Fig. 2(b). The state of each soft leg is characterized by

$$\mathbf{s} = [\alpha_b, \alpha_r, z_1]^\top, \quad (7)$$

as shown in Fig. 2(b)–(d). The mapping of a feasible $\mathbf{s}$ into the corresponding tendon displacement $\mathbf{x}_{ten}$ is investigated in the previous work [8] as the following function.

$$\mathbf{x}_{ten} = g_{inv}(\mathbf{s}) = f_{inv}([\alpha_b, \alpha_r]^\top) + \mathbf{1}z_1, \quad (8)$$

where $f_{inv} : \mathbb{R}^2 \rightarrow \mathbb{R}^3$ is derived through geometric analysis, and

$$\begin{aligned} x_A &= R_d \alpha_b \cos(\alpha_r), \\ x_B &= R_d \alpha_b \cos(\alpha_r + \frac{2\pi}{3}), \\ x_C &= R_d \alpha_b \cos(\alpha_r + \frac{4\pi}{3}). \end{aligned} \quad (9)$$

R_d is the radius of the circle formed by the three guiding holes on a disk. $\mathbf{x}_{ten}$, as the tendon displacement, is further used as the motor reference for the low-level position control. The synchronization of the designed trajectory on $\mathbf{s}$ and $\mathbf{x}_{ten}$ will lead to a gait pattern of the robot on the task level.

Moreover, a simulation environment utilizing MATLAB Simulink Multibody blocks is developed [5,8], providing a digital emulation of the real robot. The plant model within the simulator exhibits a high level of modeling accuracy corresponding to the physical world, which supports and prompts the design of motion controllers. In addition, note that various terms, e.g., real robot, physical robot, and real-world robot, are interchangeable in what follows, all referring to the entity existing physically within the tangible environment, as opposed to the simulated robotic platform employed.

4.2. Gait pattern generation

In the previous study [8], the temporal trajectory of $\mathbf{s}$ is characterized by an ad hoc design approach, which is non-optimal in terms of robot walking speed and efficiency. State-of-the-art research usually identifies the walking of a quadruped robot by rhythmic patterns; hence, the nonlinear oscillator can be used as an ideal option to model a rhythmic movement in a formalized and parametric way. Therefore, to improve the gait design, we employ nonlinear oscillators to model the rhythmic movement of the actuator states.

Using the inverse kinematic model in (8), this study applies the Hopf oscillator as a fundamental pattern generator for the robot locomotion pattern design. Following the notation in Section 3.4, we denote by $\mathbf{o}_i$ the oscillator state of cycle behavior s_i of leg- i , $i \in \mathcal{I} = \{1, 2, 3, 4\}$. To unify the cycle behavior of each leg and promote their coordination, we assume the oscillator motion of legs $\mathbf{o}_i$ is originated from a primitive oscillator $\mathbf{o}$ based on (5)–(6).

Nevertheless, the Hopf oscillators of each leg are yet coordinated to shape gait patterns and generate motor trajectory references. To realize it, coupling term $\mathbf{q}_i$ is designed to generate coupled oscillator $\mathbf{o}_i$ of each leg from the primitive oscillator $\mathbf{o}$. The distinct behavior of each leg is represented by a phase difference

$$\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3, \theta_4]. \quad (10)$$

Thus, the rhythmic trajectory of each leg is defined by

$$\mathbf{o}_i(t) = \mathbf{o}(t - \Delta t_i), \quad \Delta t_i = \theta_i / (2\pi), \quad i \in \mathcal{I}. \quad (11)$$

This study considers that the oscillator $\mathbf{o}_i$ is coupled and perturbed by the primitive oscillator $\mathbf{o}$. Given the target phase difference $\theta_i = \boldsymbol{\theta}[i]$, the coupling term of the i th oscillator is calculated by

$$\mathbf{q}_i = \begin{bmatrix} q_1 \\ q_2 \end{bmatrix}_i = \begin{bmatrix} 0 \\ \varepsilon(o_1 \sin \theta_i + o_2 \cos \theta_i) \end{bmatrix}, \quad (12)$$

where $\varepsilon > 0$ denotes the coupling strength. By designing the values of $\boldsymbol{\theta}$, the oscillator of each leg $\mathbf{o}_i$ is defined. Thus, actuators of legs can be synchronized to compose different locomotive patterns, e.g., trotting, and galloping, depending on the values of $\boldsymbol{\theta}$. Fig. 3 illustrates an example of the non-harmonic oscillator response with varying frequencies, where $\mathbf{o}'$ is generated from $\mathbf{o}$. $\mathbf{o}_1 = \mathbf{o}[1]$ and $\mathbf{o}'_1 = \mathbf{o}'[1]$ defined in (5) are used as the output of the oscillator. In the figure, two coupled oscillators have the same pattern but different phases. $\Delta t'$ is defined by $\theta' = \pi/2$.

Subsequently, the derivation of $\mathbf{o}_i$ is applied to the design of s_i . Recap that in s_i , the rotational angle $\alpha_{r,i}$ of the leg influences the movement direction of the quadruped robot. Especially, the prior study [8]

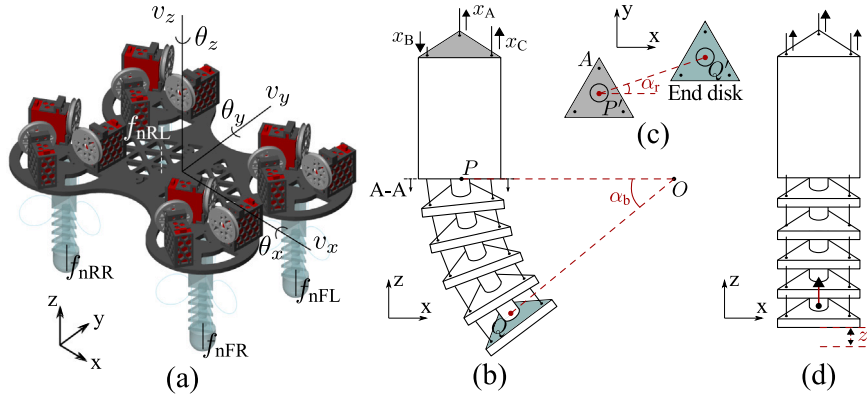


Fig. 2. Overview of the soft robot and its Tendon-driven Soft Actuator (TSA): (a) Rendered robot with key states. (b) Structure of TSA: rigid upper thigh and bendable lower section with α_b . (c) Top view of bent TSA with α_r . (d) TSA compression via equal tendon pull with z_1 .

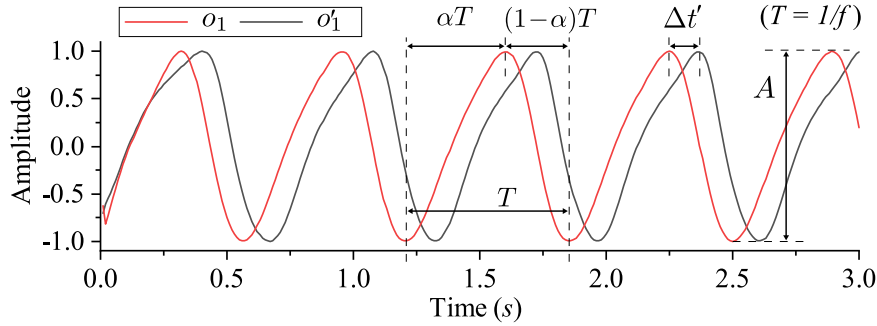


Fig. 3. The non-harmonic oscillator response with varying frequencies. In this example, the initial state is randomized, and $[A, f, \alpha, \tau, k, \epsilon, \Delta t'] = [1, 1.33, 0.75, 1, 1000, 5, 0.25]$.

has demonstrated that the direction of movement θ_z corresponds to the angular rotations of the legs when all four legs exhibit uniform values, i.e., $\forall i \in I, \alpha_{r,i} = \theta_z$. Therefore, given a reference trajectory with the direction of movement θ_{ref} , we can simplify the problem by setting $\alpha_{r,i} = \theta_{\text{ref}}$.

Thus, the cyclic behavior s_i is essentially defined by $\alpha_{b,i}$ and $z_{1,i}$, which determine the trajectory and amplitude of the foot pattern. Based on the previous analysis, the rhythmic behaviors can be effectively characterized by the state output $o_{1,i}$ originating from the Hopf oscillator. Note that, for the sake of simplifying the design process and limiting the parameter space, we consider $\alpha_{b,i}$ and $z_{1,i}$ as interrelated, and both are characterized by the same oscillator output $o_{1,i}$ but with distinct phases and gains, i.e.,

$$\alpha_{b,i}(t) = o_{1,i}(t), \quad \frac{\alpha_{b,i}(t)}{A_{\alpha_b}} = \frac{z_{1,i}(t - \varphi)}{A_{z_1}}, \quad (13)$$

where $\varphi \in [0, T]$ is a design parameter, which represents the delay between the patterns of $\alpha_{b,i}$ and $z_{1,i}$. This delay parameter allows for fine-tuning the synchronization between these two critical components in the gait pattern generation. A_{α_b} and A_{z_1} denote the amplitudes for α_b and z_1 , respectively. We consider that both trajectories share the same frequency f and shape ratio α in order to generate the rhythmic pattern.

4.3. Optimization problem formulation

Based on the above analysis, the coordinated cycle behavior of each leg s_i can be defined by the oscillator state $\mathbf{o}_i$, the correlated phase difference φ , and the reference direction θ_{ref} , and $\mathbf{o}_i$ is further influenced by the base oscillator $\mathbf{o}$ and coupling term $\mathbf{q}_i$ through (11)–(13). Thus, $s_i = f_{\text{CPG}}(\mathbf{o}, \mathbf{q}_i, \varphi, \theta_{\text{ref}})$.

$$(14)$$

Eq. (14) provides a formulation of the rhythmic and coordinated gait pattern enabled by the Hopf oscillator. This formulation enables the

generation of coordinated leg movements in response to the specified reference direction and desired phase synchronization. Accordingly, the references $\mathbf{x}_{\text{ten}}$ for the 12 motors can be calculated from s_i through (8). If the reference trajectories $\mathbf{x}_{\text{ten}}$ are provided to the robot's simulation model, the simulation outcomes can be analyzed to evaluate the quality of the gait pattern.

Generally, the gait of the legged robot is determined by three factors: stride, duty cycle, and phase factor. In our model, the phase factor is controlled by θ in (10) for the leg synchronization, and the other two are derived from the trajectory of s_i based on (14). Note that in (14), given the initial state $\mathbf{o}(t_0)$, $\mathbf{o}$ is determined by parameters $A_{\alpha_b}, A_{z_1}, f, \alpha, \tau, k$, and $\mathbf{q}_i$ is computed based on ϵ, θ_i . Among those, τ, k, ϵ control the stability and convergence of the oscillators but will not influence their steady-state behaviors; thus, those parameters are irrelevant to the gait design, which are regarded as constant and predefined in the design phase.

Therefore, besides θ_{ref} and θ that explicitly define the robot walking direction and phase factor (which can be pre-designed), the other walking properties are determined by $\mathbf{o}$, $\mathbf{q}_i$ and φ . To summarize, based on (5), (6), and (13), the choice of parameters in the vector $\mathbf{p} = [A_{\alpha_b}, A_{z_1}, f, \alpha, \varphi]^T \in \mathbb{R}^5$ significantly influences the stride and duty cycle of the gait, thus influences various aspects of the robot's walking properties, e.g., walking speed, Cost of Transport, terrain clearance, and stride length.

By designing and optimizing the associated design parameters in $\mathbf{p}$, the parametric formulation of the gait pattern s_i enables the shaping of an optimal robot walking pattern. Considering the inverse kinematic mapping in Section 4.1, the relationship between $\mathbf{p}$ and the objective function set up an end-to-end design from motor actuation commands to the robot walking task-level control. Specifically, as an intuitive representative of the interested properties, this study defines the objective function by maximizing the robot walking speed v . Nevertheless, the

proposed approach can also be applied and analyzed for the other walking properties as mentioned before. Within our CPG-based parametric gait pattern framework, we assume that a set of parameter $\mathbf{p}$ will lead to a stabilized robot walking speed v .

Owing to the lack of a known mathematical form, the evaluation of the walking speed v for a candidate parameter vector $\mathbf{p}$ is based on the simulation results of the robot model. If we view the simulated robot as a black-box model, this parameter design problem becomes a black-box optimization problem. Given the unknown mapping function $J : \mathbb{R}^5 \rightarrow \mathbb{R}$ with the design space $\mathcal{P} = \{\mathbf{p} \in \mathbb{R}^5 : \underline{p}_i \leq p_i \leq \overline{p}_i\}$, where $\underline{p}_i$ and $\overline{p}_i$ denote the lower and upper bounds for each element, the optimal walking speed can be identified through

$$v^* = \max_{\mathbf{p} \in \mathcal{P}} J(\mathbf{p}), \quad (15)$$

This black-box optimization problem is solved by the Bayesian optimization method introduced in Section 3.3.

5. Adaptive optimal gait design by multi-fidelity Bayesian optimization

5.1. Reality gap from the digital replica

The optimization problem in (15) aims to identify the optimal parameter vector $\mathbf{p}$. To address this, we employ BO with GP as a surrogate model to approximate the unknown objective function $J(\mathbf{p})$, which represents the robot's walking performance. The GP model, introduced in Section 3.2, is specified by a mean function $m(\cdot)$ and a covariance function $k(\cdot, \cdot)$, which allow the model to predict the performance at new points in the parameter space based on previous observations. The optimization of these parameters is treated as a black-box problem, where the objective function $J(\mathbf{p})$ does not have an explicit form but can be evaluated through simulations or physical experiments. It can be effectively addressed by the BO method as introduced in Section 3.3. BO iteratively selects candidate points for evaluation and uses the gained information to guide the search towards the optimal solution. However, implementing this process in a physical robotics setup is typically troublesome, since each direct evaluation of $J(\cdot)$ on the robot can be resource-intensive, tedious, and risky for the robot.

To minimize the number of physical evaluation queries and optimize the parameters efficiently, we leverage a simulator model of the quadruped robot [5]. This simulator model serves as a digital replica of the real robot, providing evaluation results with reduced effort and time compared to physical evaluations. Consequently, the optimal combination of parameters can be identified through the optimization of the evaluation results obtained via simulation, making the process more cost-effective.

Nonetheless, it is crucial to account for the unavoidable discrepancies between the simulator and the real robot, and the non-negligible reality gap is demonstrated in our previous experiments [5]. As an alternative, the optimal points can be identified directly from the physical robot with the BO framework; however, as previously discussed, $J(\cdot)$ is expensive to evaluate in the physical experiments, resulting in increased workload and difficulty in data collection during the iterative process.

To ensure that optimal performance can be achieved in the physical environment, we introduce an MFBO approach based on the BO framework for domain adaptation and online training. It facilitates the transfer of knowledge from simulation to reality during parameter optimization. Thus, the optimal solution is adapted and evolved efficiently from model-based optimization to physical implementation.

5.2. Multi-fidelity Bayesian optimization

Both the simulator and the physical robot can be modeled as two different GP models with the same input and output states. Thus, in the BO framework, the two GP models for the same subject are correlated, and their inter-model correlations can be exploited for efficient learning. To achieve this, we employ the MTGP model to substitute the single output GP model. This substitution allows us to leverage multiple outputs that capture diverse behaviors of various plant models. We create a tuple $\langle \mathbf{p}_i, j \rangle$ by augmenting the input sample with the fidelity identifiers j , where $j \in \{1, 2\}$ denotes the simulator and physical environment. To reflect the similarity constraint, we define a new covariance function over the identifier by $k_f(j, j')$. With intrinsic model of coregionalization [57], we reconstruct the kernel function with

$$\tilde{k}(\langle \mathbf{p}_i, j \rangle, \langle \mathbf{p}_{i'}, j' \rangle) = k_f(j, j')k(\mathbf{p}_i, \mathbf{p}_{i'}), \quad (16)$$

where $k(\cdot, \cdot)$ is defined in Section 3.2. $k_f(\cdot, \cdot)$ is the additional covariance function representing the correlation over models, which takes the model labels as input.

For the training input set $\mathbf{P}$ with n samples, we first assume the complete set of evaluation results on $J(\cdot)$ is available for both fidelity levels with $\tilde{\mathbf{v}} = [\mathbf{v}^1, \mathbf{v}^2]$, where $\mathbf{v}^j = [v_1^j, \dots, v_n^j]$, $j \in \{1, 2\}$. Based on (16), the covariance matrix $\tilde{K}$ for the data from both fidelity levels can be obtained with

$$\tilde{K} = K_f \otimes K_p, \quad (17)$$

where $\otimes$ denotes Kronecker product, $K_f \in \mathbb{R}^{2 \times 2}$ is a symmetric matrix whose diagonal entries represent the inner-task relationship with themselves, and the other elements describe the correlations of different fidelity levels. Note that if all the off-diagonal entries are zeros, K_f becomes an identity matrix, which means all the underlying data sources are independent.

The parameters for an MTGP model include $\boldsymbol{\lambda}$ and K_f . We denote them collectively as $\Lambda = [\boldsymbol{\lambda} \quad \text{vec}(K_f)]^\top$, where $\text{vec}(\cdot)$ denotes the column vectorization of a matrix. An MTGP model is optimized similarly to that in Section 3.2 by maximizing the marginal likelihood for Λ . We leave out the derivation of hyperparameter learning for simplicity. Accordingly, the prediction of the test point $\langle \mathbf{p}', j' \rangle$ can be achieved by calculating the conditional probability with

$$p(v' | \langle \mathbf{p}', j' \rangle, \Lambda^*, \mathbf{P}, \tilde{\mathbf{v}}) = \mathcal{N}(\mu_{\mathbf{p}'}, \sigma_{\mathbf{p}'}), \quad (18)$$

and

$$\begin{aligned} \mu_{\mathbf{p}'} &= \tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle)^\top \tilde{K}^{-1} \tilde{\mathbf{v}}, \\ \sigma_{\mathbf{p}'} &= \tilde{k}(\langle \mathbf{p}', j' \rangle, \langle \mathbf{p}', j' \rangle) - \tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle)^\top \tilde{K}^{-1} \tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle), \end{aligned} \quad (19)$$

where $\tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle)$ denotes the kernel vector in the MTGP model for the test point tuple $\langle \mathbf{p}', j' \rangle$, and

$$\tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle) = (K_f)_{j'} \otimes k(\mathbf{P}, \mathbf{p}'). \quad (20)$$

$(K_f)_{j'}$ is a 2×1 vector where $(\cdot)_{j'}$ selects the j' -th column of the matrix K_f . Recall that $k(\mathbf{P}, \mathbf{p}')$ is an $n \times 1$ vector, which leads to the size of kernel vector $\tilde{k}(\mathbf{P}, \langle \mathbf{p}', j' \rangle)$ becoming $2n \times 1$. It evaluates the kernel value of the test input with respect to all training data points from both fidelity models. To differentiate the single output GP model in (1)–(2), we denote by $\tilde{g}(\langle \mathbf{p}, j \rangle)$ the MTGP model.

Built upon the MTGP model, the MFBO process can be updated accordingly. To be specific, in (3)–(4), $g(\mathbf{p})$ is evaluated instead by $\tilde{g}(\langle \mathbf{p}, j \rangle)$ through (16)–(20). And the generated point by maximizing the acquisition function corresponds to an optimal candidate point for the physical robot. By performing the MFBO iterations, the precision of the MTGP is enhanced by incorporating additional data from the physical measurement. Consequently, this process facilitates the identification of the optimal point for the physical robot, thereby enabling the achievement of domain adaptation across models efficiently.

Remark 1. Owing to the expression simplicity, we utilize the isotopic data [57] for the derivation in (17)–(20), where the outputs $\tilde{\mathbf{v}}$ are assumed fully observable for both simulator and physical robot based on the input set $\mathbf{P}$. However, in practical implementation, the collected data $\tilde{\mathbf{v}}$ cannot be uniform among different fidelity levels due to the extraordinary cost of the physical experiments. It leads to the heterotopic data scenario, where some of the outputs from the physical environment are unavailable. Nonetheless, the derivation still holds with the heterotopic data, but the covariance matrix $\tilde{\mathbf{K}}$ is inappropriate with matrix-wise calculation, as some elements in $\mathbf{K}_{\mathbf{p}}$ of (17) correspond to un-examined data and will not be observed. Alternatively, if the number of data is n^j for the j th data source, the matrix size will be changed, and $\tilde{\mathbf{K}}$ becomes an $N \times N$ matrix, where $N = n^1 + n^2$. Each entry in $\tilde{\mathbf{K}}$ can still be obtained by using (16) through element-wise calculation. Eqs. (18)–(20) will change accordingly, where $\tilde{\mathbf{k}}(\mathbf{P}, \langle \mathbf{p}_i, j \rangle)$ becomes a vector of size $N \times 1$.

The MFBO approach can be outlined as a two-stage learning process, as summarized in Algorithm 1. First, we initialize a training data set by randomizing a set of parameter candidates $\mathbf{P}_{\text{init}}$ and measuring their output $\mathbf{v}_{\text{init}}$ through simulation. These parameters and measurements construct the data set for the initial GP model training. Subsequently, by running the traditional BO on the simulation platform (with the fidelity identifier $j = 1$) in an iterative way, we generate an optimal parameter vector $\mathbf{p}_{j=1}^*$, along with the accumulation of data $[\langle \mathbf{p}_i, j = 1 \rangle, v_i]$. After that, we leverage $\mathbf{p}_{j=1}^*$ as the starting point and perform MFBO on the physical experiment platform (with the fidelity identifier $j = 2$). Following the evaluation of each candidate point $\mathbf{p}_k$, we augment the data set with a new tuple $\langle \mathbf{p}_k, j = 2 \rangle$ and its corresponding result v_k . The optimal parameter $\mathbf{p}_{j=2}^*$ is determined upon the completion of iterations in the MFBO process.

Importantly, by using the MFBO approach in Lines 12–23, the amount of information gained from both sources with different fidelities can be used and contribute to the optimization process on the real robot, which saves a significant overhead by avoiding re-exploring the same parameter space on the physical platform. Owing to the computational efficiency and accuracy of the simulation platform, our MFBO approach yields substantial improvements in terms of the speed of optimization.

6. System architecture overview

To run the MFBO algorithm on the physical robot, the algorithm requires either real-time access to the robot simulation model or the storage of the training data for solving the BO problem of (15). Either is a big challenge to the training efficiency and the real-time control of the physical robot. To alleviate the data storage and computational burden on the robot's onboard microprocessor, we employ an edge computing framework to offload the MFBO training process to an edge server.

The control system architecture of the soft quadruped robot is illustrated in Fig. 4. The robot's onboard computation task mainly focuses on the sensor data collection and control signal actuation. This corresponds to a low level of decentralization (sometimes referred to as a distributed I/O approach). Three types of sensors are integrated into the robot, including the Inertial Measurement Unit (IMU), Time of Flight (ToF), and force sensors. Given the study's emphasis on walking speed, both IMU and ToF sensors are employed to estimate longitudinal velocity, while force sensors only contribute to footfall pattern analysis. The onboard collected data from the sensors are sent to the edge server for the state estimation of the robot's velocity. Subsequently, the robot receives control signals from the edge server to govern soft leg actuation. Actuation is executed through position control of the servo motors, with lower-level PD controllers enhancing stability and efficiency.

The edge server is dedicated to executing computationally and memory-intensive tasks, encompassing functions such as state estimation, MFBO training, CPG oscillator calculation, and kinematics

Algorithm 1: Multi-fidelity Bayesian optimization

Data: Initial set $\mathbf{P}_{\text{init}}$, design space bounds $[\mathbf{p}_i, \overline{\mathbf{p}}_i]$, initial guess of K_f , number of iterations $i_{\text{max}}, k_{\text{max}}, i = k = 1$

Result: Optimal parameter on real robot $\mathbf{p}_{j=2}^*$

- 1 Evaluate $\mathbf{P}_{\text{init}}$ on the simulator and get $\mathbf{v}_{\text{init}}$;
- 2 Build $\mathcal{T} \leftarrow \{[\langle \mathbf{p}_i, j = 1 \rangle, v_i] \mid \mathbf{p}_i \in \mathbf{P}_{\text{init}}, v_i \in \mathbf{v}_{\text{init}}\}$;
- 3 Train GP model $g(\cdot)$ from $\mathcal{T}$ with parameter λ ;
- 4 **while** $i < i_{\text{max}}$ **do**
- 5 Perform BO based on $\mathcal{T}$ and λ , get new candidate point $\mathbf{p}_i$;
- 6 Evaluate $\mathbf{p}_i$ on the simulator and get v_i ;
- 7 Update set: $\mathcal{T} \leftarrow \mathcal{T} \cup \{[\langle \mathbf{p}_i, j = 1 \rangle, v_i]\}$;
- 8 Update the GP model $g(\cdot)$ and get a new λ ;
- 9 Update iteration: $i \leftarrow i + 1$;
- 10 **end**
- 11 $\mathbf{p}_{j=1}^* \leftarrow \mathbf{p}_i, i = \arg \max_i (v_i), [\langle \mathbf{p}_i, j = 1 \rangle, v_i] \in \mathcal{T}$;
- 12 **while** $k < k_{\text{max}}$ **do**
- 13 **if** $k = 1$ **then**
- 14 $\mathbf{p}_k = \mathbf{p}_{j=1}^*, \Lambda = [\lambda \quad \text{vec}(K_f)]^T$
- 15 **else**
- 16 Perform MFBO based on $\mathcal{T}$ and Λ , get a new candidate point $\mathbf{p}_k$;
- 17 **end**
- 18 Evaluate $J(\mathbf{p}_k)$ on the real robot and get v_k ;
- 19 Update set: $\mathcal{T} \leftarrow \mathcal{T} \cup \{[\langle \mathbf{p}_k, j = 2 \rangle, v_k]\}$;
- 20 Update the MTGP model $\tilde{g}(\cdot)$ and get a new Λ ;
- 21 Update iteration: $k \leftarrow k + 1$;
- 22 **end**
- 23 $\mathbf{p}^* \leftarrow \mathbf{p}_k, k = \arg \max_k (v_k), [\langle \mathbf{p}_k, j = 2 \rangle, v_k] \in \mathcal{T}$;

conversion. All functionalities on the edge server are deployed within a virtual machine operating in the Ubuntu 20.04 environment. The raw data of sensor feedback from the soft quadruped robot, including quaternions and linear accelerations, distances to surroundings, and contact forces, are fused to enhance speed estimation through a Kalman filter. The resultant stabilized speed signal serves as the input for MFBO training, facilitating the optimization of parameters for the CPG controller. Subsequently, the oscillator signal is updated, and actuation commands are derived through kinematics conversion. The communication infrastructure between the robot's controller and the edge server relies on 5G technology, facilitated by the Quectel RM500Q-GL 5G HAT integrated into the soft robot. The employment of 5G communication not only provides improved reliability and lower latency for the real-time control task, but also enhances mobility support with expanded coverage, which makes it particularly advantageous for potential industrial IoT use cases with soft robots.

7. Results and validation

7.1. Baseline selection

To assess the effectiveness of the implemented BO approach during the training phase, we conduct performance comparisons with the following baselines. These baselines include algorithms/search methods previously introduced in Section 1, which serve as a reference for state-of-the-art approaches in the domain.

- Adaptive grid search (AGS) [58]: Owing to the exponential complexity associated with grid search at a finer granularity for sampling, the AGS approach is used to avoid exhaustive search. AGS iteratively refines the search space based on promising regions from the previous optimal solution. We consider the point identified by AGS as the global optimum.

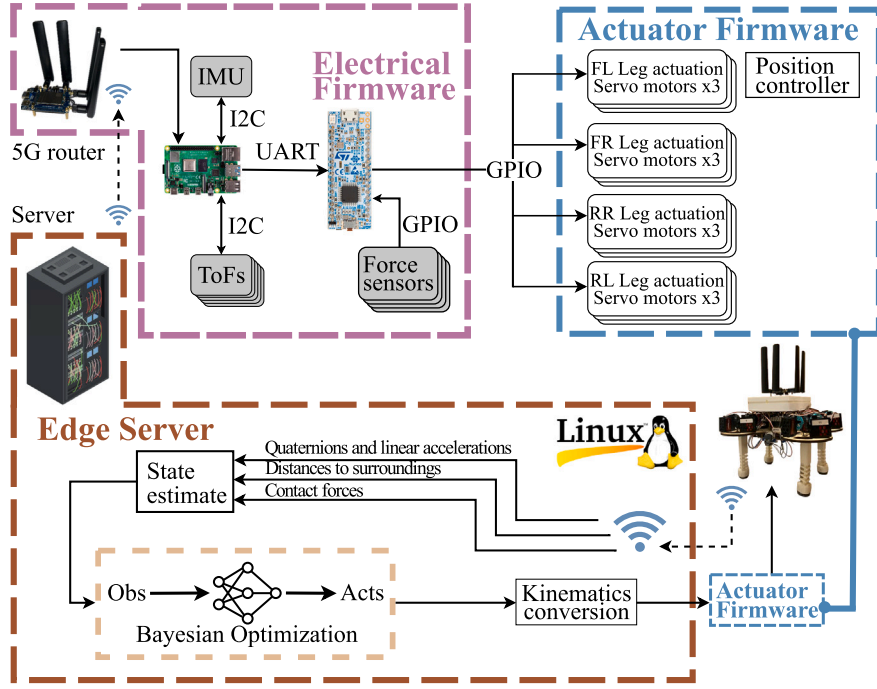


Fig. 4. Control system architecture. The actuator and electrical firmware are physically integrated into the robot. The unidirectional arrows surrounding radio waves denote the transmission and reception of messages between soft robots and the edge server.

- Simulated annealing (SA) [59]: SA is utilized as the benchmark for black-box optimization comparison due to its effectiveness as a single-point search algorithm. Note that population-based algorithms, e.g., Genetic Algorithms (GA) [43] and Particle Swarm Optimization (PSO), are not ideal due to the computational expense of evaluating a substantial amount of samples per iteration.
- Random search (RS) [60]: RS represents a worst-case scenario without intelligent exploration, which is used to quantify the improvement of the other methods.
- Omni-directional walking design (OWD) [8]: OWD is proposed in the previous work for the soft robot walking gait design. It is regarded as a baseline in terms of ad hoc design without optimization or search processes; thus, it is only used to compare the performance of the identified optimal gait pattern.

When selecting the baseline methods, we consider several grid-based methods, including standard grid search, exhaustive grid search, and non-equally spaced grid search [61]. Exhaustive grid search is impractical for our continuous design parameters due to the sheer number of evaluations it requires. Standard grid search, while feasible, has limitations in resolution, where it either risks missing optimal solutions with a coarse grid or wastes computational resources with a fine grid. Non-equally spaced grid search, which focuses on grid points with varying densities in key areas, typically requires prior knowledge of the problem and can be less adaptable in unknown search spaces. Given these limitations, we selected Adaptive Grid Search (AGS), which dynamically refines the grid based on previous evaluations. Additionally, we excluded population-based algorithms such as GA and PSO due to the high computational costs associated with our accurate but time-intensive soft robot simulation model. Each physical second requires approximately 30 s of computation. Population-based algorithms evaluate multiple solutions per iteration (typically 50 to 100 solutions per generation over hundreds of generations), which would result in excessive computational time. For this reason, we deemed such algorithms infeasible within the reasonable time constraints of our optimization process.

Note that, except OWD, the other baselines are solely employed for evaluating the optimization performance at the simulation level, as

they cannot effectively control the physical robot to walk. Except AGS, each search method is assessed through 3 evaluations using distinct random seeds. We impose a predefined limit of 300 iterations as a stopping criterion for SA, RS, and BO. Specifically, we focus on the following key performance indicators for the black-box optimization alternatives comparison:

- Gait Quality: Measured by the robot's walking speed (v), which directly reflects the objective function and effectiveness of the optimized gait parameters. A higher stable walking speed, without causing the robot to fall or lose balance, indicates a higher-quality gait.
- Convergence Speed: The number of iterations or evaluations required for each algorithm to reach a near-optimal solution. This metric is critical as it directly relates to how quickly the algorithm can find an effective solution for optimizing the robot's gait.

The selection of the surrogate model in BO is crucial in identifying optimal points of design parameters. As commented in Section 3.3, various surrogate models may be employed for BO. Specifically, we assess the performance of these surrogate models against GP, which are summarized and outlined in a previous study [54]:

- Random forest (RF) [62]: RF is an ensemble method consisting of multiple regression trees. Each tree is trained on a bootstrap sample of the training data, and the final prediction is obtained by averaging the predictions of all trees.
- Random Fourier Expansion (RFE) [63]: RFE generates random features using Fourier basis functions and trains a linear regression model on these features to approximate the objective function.
- Support Vector Machine (SVM) [64]: SVM finds the best hyperplane to separate different classes by maximizing the margin between support vectors. We use Radial Basis Function (RBF) as the kernel function.
- Piece-wise Linear (PL) [65]: PL surrogate model uses rectified linear units (ReLUs) in a linear combination, where the ReLU input represents the linear mapping of the input data [65].

Table 2
Comparison of the optimization performance.

	AGS	SA	RS	BO
Max speed (m/s)	0.312	0.264	0.125	0.286
Optimization time (h)	72.367	5.025	4.567	1.278

The hyperparameters in the surrogate models are configured in alignment with the studies referenced by Blicke et al. [54]. All the simulation evaluations are executed on a desktop with Intel(R) Core(TM) i7-8700 CPU at 3.20 GHz, six cores, and 8.0 GB installed memory (RAM).

7.2. Comparison of optimization methods

Table 2 lists the optimization results by different methods with the associated computational time. Note that this comparison is conducted with the simulation platform. Among all the methods, AGS delivers the best performance with the global optimum speed at 0.312 m/s. Nevertheless, it comes at a significant computational time cost, consuming over 72 h as it evaluated 3072 points within 3 iteratively refined parameter regions. In contrast, the BO approach only takes 1.278 h to converge after roughly 70 iterations, reaching an optimal speed of 0.286 m/s, which corresponds to 91.67% of the global optimum. In addition, SA does not meet the objective function tolerance criteria and gets terminated at 300 rounds. It results in an identified optimal point of 0.264 m/s in a duration of 5.025 h, which is slightly worse than BO. On the other hand, RS provides a lower bound reference of employing the defined CPG oscillator-based gait pattern. It offers a baseline speed of 0.125 m/s and consumes 4.567 h to finish the 300-iteration search. In comparison to AGS and RS, the application of the BO approach demonstrates its effectiveness in exploring the optimal gait pattern for soft robots. Additionally, in the context of black-box optimization, it also exhibits superior efficiency compared to SA. Moreover, the underlying GP model in BO facilitates domain adaptation for subsequent sim-to-real transfer, which is a feature not achievable by SA.

For a more comprehensive analysis of the robustness of BO, Fig. 5 illustrates the BO training performance across different trials with randomly initialized seeds and noise, where the corresponding parameter setting is given in Table 4. Note that the variable ranges are defined with the same extreme values as deployed in the study [8] for a fair comparison. It can be seen that the robot stays at an exploration phase within the design space for the first 40 iterations, followed by a noticeable performance improvement for the next 20 iterations. Subsequently, convergence is achieved with a stable speed at around 70 iterations. In the simulated environment, the robot ultimately learns a stable speed of approximately 0.28 m/s. This represents a remarkable improvement compared with the previous study [8] (0.14 m/s). The optimized gait pattern within the identical motion space leads to a twofold increase in walking speed.

7.3. Comparison of surrogate models

Table 3 compares the performance of different surrogate models. We construct a test data set by pseudo-randomly selecting 1000 samples in order to cover the entire design search space. We evaluate the goodness-of-fit of models with R^2 and NMSE to quantify the prediction accuracy. To evaluate the generalization and robustness of the model, we conduct the experiments through a 5-fold cross-validation by partitioning the dataset into five subsets, using each subset once as a validation set, and averaging the performance metrics across all folds. Hence, we present the mean R^2 and its standard deviation from cross-validation, along with the mean NMSE. In addition, we evaluate the computational time for each surrogate model regression to assess their computational complexity.

Table 3
Cross-validation for comparing the surrogate models.

	RF	RFE	SVM	PL	GP
Mean R^2	0.9373	0.8880	0.8902	0.9294	0.9856
Standard deviation	0.0121	0.0262	0.0220	0.0140	0.0049
Mean NMSE	0.1134	0.1519	0.1487	0.1201	0.0478
Computational time (s)	0.0546	0.1584	0.1180	10.9937	4.7888

Table 4
Parameter setting for BO training.

Parameters	Values
$ \mathbf{P}_{init} $	10
$\bar{\mathbf{p}}$	$[0.7, 0.008, 2, 0.9, 2\pi]^T$
$\underline{\mathbf{p}}$	$[0.1, 0.001, 0.4, 0.1, 0]^T$
i_{max}	70

In Table 3, it is evident that under a relatively small dataset scenario, the employed GP surrogate model exhibits a noticeable advantage over other methods in terms of modeling accuracy. Furthermore, during the cross-validation experiment, it demonstrates the lowest standard deviation value, indicating stable out-of-sample performance and strong generalization to unseen data. On the other hand, the GP method has larger computational complexities, which grow cubically and quadratically with the number of training points during model training and online inference, respectively. In comparison to other benchmark methods, it only shows better computational efficiency against PL, which requires back-propagating training for ReLUs. This result gives us insights into the trade-off between model accuracy and real-time efficiency when employing the GP model, thereby emphasizing the significance and justification of our edge off-loading architecture.

7.4. MFBO and physical deployment

This section addresses the results of MFBO during the physical deployment, and the training process is depicted in Fig. 6. As previously noted, we initiated the MFBO training with the optimal gait discovered in the simulation environment. Due to model errors, the robot's actual walking behavior differs from the simulated speed. Specifically, it struggles to attain the desired speed and only reaches approximately 0.025 m/s.¹ Nevertheless, after several iterations of MFBO training, the robot rapidly adjusts the walking pattern to suit the physical environment. Consequently, its walking speed gradually increases to around 0.2 m/s and reaches a saturation point. Despite some oscillations and fluctuations in the training curves across 5 different trials attributable to environmental noise, they exhibit a consistent overall trend and stabilize within 10 iterations.

Furthermore, the robot's walking speed at the optimized point in the physical environment is plotted in Fig. 7. As before, we conduct 5 trials and provide both individual trial results and the average performance. The robot accelerates to a stable speed within approximately 1.2 s. In particular, the largest speed goes up to 0.344 m/s during the testing with an average stabilized speed of 0.214 m/s. This represents a significant improvement of 52.71% compared to the physical testing results of the OWD solution within the same parameter range. Additionally, compared to OWD, our approach overcomes the limitations of static parameters by enabling real-time adaptability and optimized performance.

Fig. 8 plots and compares the foot trajectories between the MFBO and OWD gaits. The MFBO controller demonstrates a substantially increased swing distance in the direction of motion, resulting in an approximately twofold expansion of the horizontal distance compared

¹ The corresponding motion patterns are viewable in the attached video.

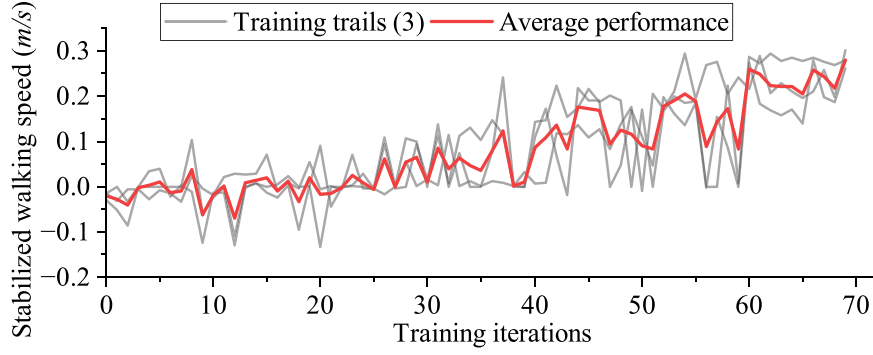


Fig. 5. BO training on simulation platform.

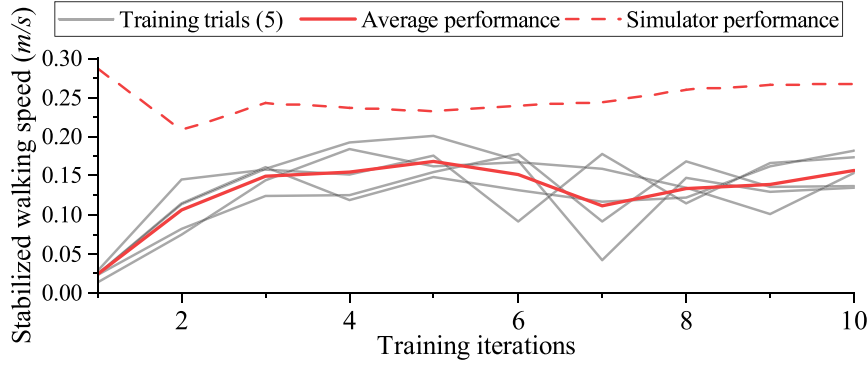


Fig. 6. MFBO physical training.

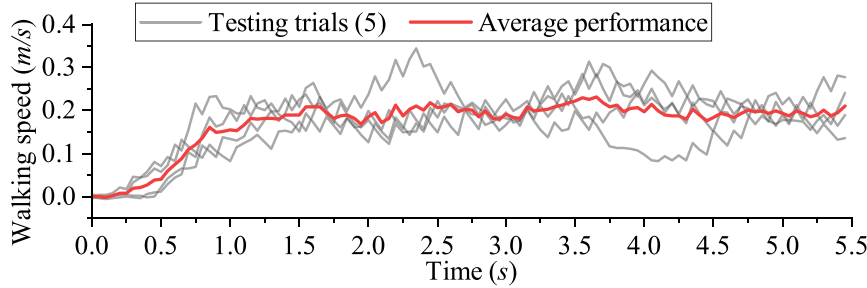


Fig. 7. Walking test of the optimized parameters.

to the OWD foot pattern. In addition, both foot patterns attain a similar maximal foot height. Note that the design space of the leg's state s remains consistent across both methods. However, the swing trajectories of α_b and z_l are different for the two methods during the design phase. This disparity leads to distinct phase synchronization behaviors between α_b and z_l , influencing the maximal horizontal distances and subsequently impacting walking patterns and speeds.

In addition, as shown in Fig. 9, a comparison of oscillator behaviors is made between the optimal point identified in the simulator and the physical environment. According to (14), the oscillator exhibits varying characteristics depending on the design parameter vector $\mathbf{p}$. Relative to the optimized gait observed in the simulator, the oscillator associated with the optimal gait in the physical test exhibits a comparatively lower frequency value f and a higher value for the shape ratio α . These differences suggest a more prompt transition between the two swing phases during physical training. The maximal bending angle α_b remains consistent across both data sources; however, in the physical testing scenario, the robot's leg requires less time for the swing-back motion and tends to exert a greater force during the rearward push against the ground. This phenomenon corresponds to the previously discussed

reality gap in Fig. 6, as the robot encounters challenges in generating sufficient power to facilitate forward movement when operating with the optimal gait derived from the simulator. Consequently, it shows more aggressive behavior by shortening the duration of the swing-back phase. Moreover, in Fig. 9(b), z_l exhibits a noticeable decrease accordingly, which is inferred to compensate for the alteration in α , since keeping the same amount of z_l with a larger α will challenge the motor response capacity. Also, increasing α without decreasing z_l will result in an overly aggressive gait pattern, potentially leading to kinetic friction and causing slippage.

7.5. Off-loading performance comparison

To highlight the necessity of edge computing architecture to realize the task off-loading process, we conducted a comparative analysis of CPG calculations executed on the robot's control board and the edge server. Note that the MFBO training part is exclusively offloaded to the edge server and is not considered in this comparison due to its excessive computational resource demands on the robot's board. Owing to the contention of computational resources in a multi-thread setup, control

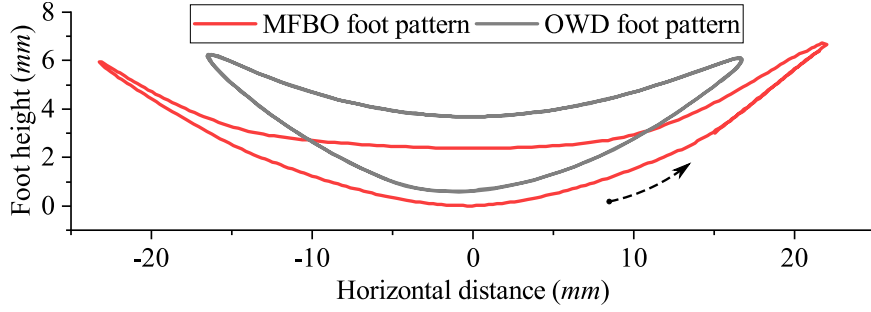


Fig. 8. Comparison of the foot pattern.

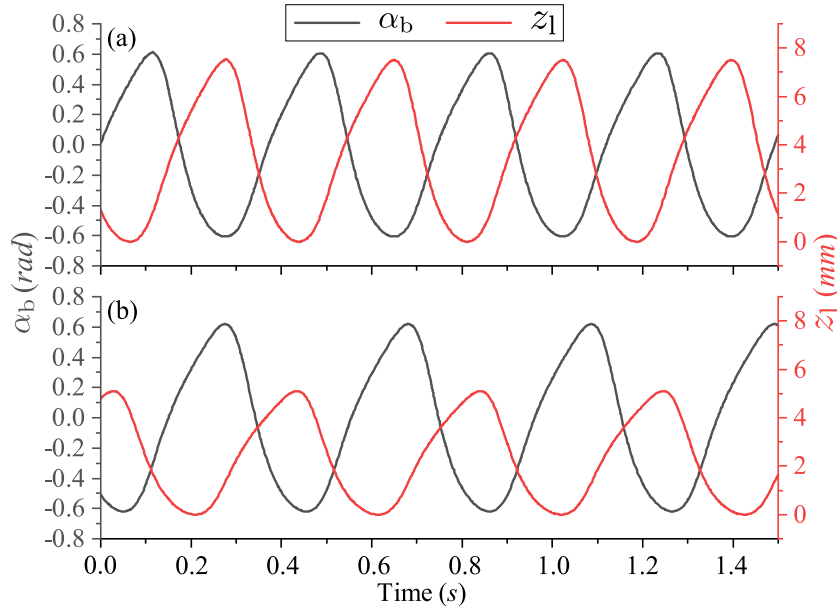


Fig. 9. Comparison of behaviors of optimized oscillators. (a): Simulation platform, (b): Physical testing platform.

tasks on the robot's onboard controller may become overdue, resulting in missed deadlines. Therefore, we evaluate the overdue status of the control tasks.

As shown in Fig. 10(a), when the control task is offloaded, the computation of the CPG actuation signals is accelerated by leveraging the advanced computation capacity of the edge server. In this scenario, 5G communication contributes to a larger portion of the total delay, which is primarily limited by the fixed proximity of the edge server and 5G bandwidth availability. The resulting end-to-end latency satisfies the defined control frequency (20 Hz), thus enabling a smooth gait actuation in the physical deployment, with average and maximum delays of 28.215 ms and 42.779 ms, respectively. Conversely, executing the computation task on the onboard controller leads to longer execution time delays, and some jobs miss the deadline. We profile the overload of the onboard computation from a thread in the lower-level controller in Fig. 10(b) and evaluate the performance influence. It is noteworthy that the plotted time consumption in Fig. 10(b) exclusively reflects the execution of the control task. However, some instances of missed deadlines, as observed in the figure, occur despite computation delays below 50 ms. This phenomenon is attributed to the incomplete execution of perception and actuation tasks within the closed-loop control period. Thus, the degradation in real-time performance results

in an inconsistent and incoherent gait pattern, with the robot getting stuck in certain postures due to missed actuation signals².

Additionally, we measure the CPU and memory usage during the execution of the CPG calculations by using the profiling tools. When running the tasks onboard, the CPU utilization of Raspberry Pi consistently peaks at 85%–95%, especially during the training process, while memory usage reaches 58% of the available resources. In contrast, when offloading the tasks to the edge server, the CPU load on the robot's controller drops to an average of 15%, and memory usage decreases to 42%, allowing more resources to be allocated to other critical functions like sensor data processing and low-level control. These quantitative metrics demonstrate a 70% reduction in CPU load and a 16% reduction in memory usage, providing stronger evidence of the system's performance gains when tasks are offloaded to the edge server.

7.6. Domain adaptation and reality gap

As discussed in Section 5.2, the domain adaptation of MFBO is realized by determining the matrix K_f with physical experiments, and

² The comparison of real-time control performance is shown in the video attachment.

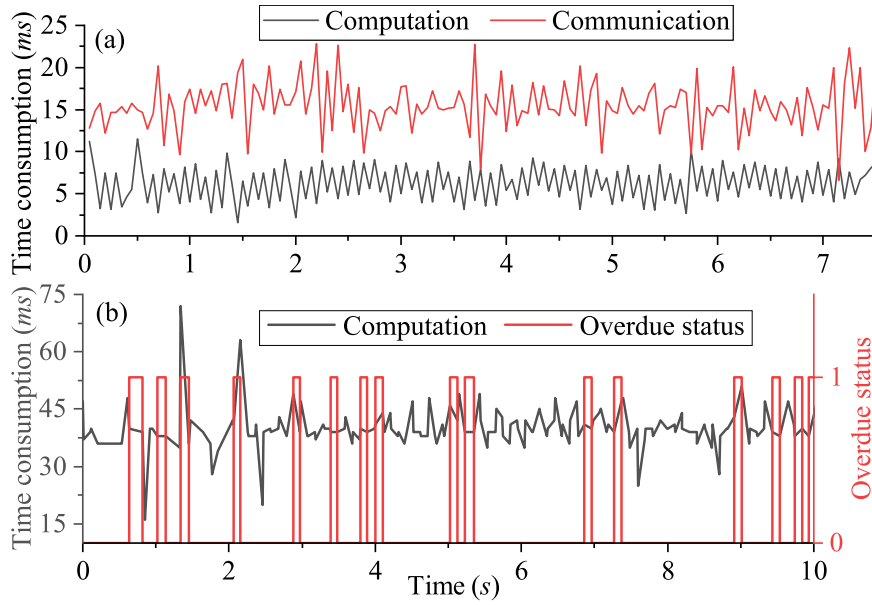


Fig. 10. Comparison of the time consumption of real-time control when the CPG is calculated at (a): edge server, (b): robot's onboard controller.

it represents the correlation between the simulator and the physical robot. To increase its interpretability, a transformation is performed on K_f by diagonal elements normalization [66]. We refer to the normalized matrix as K'_f , and $K'_f = \begin{bmatrix} 1 & 0.575 \\ 0.575 & 1 \end{bmatrix}$. The diagonal elements of K'_f are unit values to eliminate the influence of label-specific scaling. The off-diagonal elements indicate that the dynamic properties of the simulator and the robot have a moderately positive correlation.

In addition, we perform a bi-directional comparison to verify the reality gap, where the performance of the optimized parameters in different MFBO iterations from a training trail is cross-validated in the simulator. The corresponding velocities are plotted in Fig. 6. Notably, in the simulated platform's initial iteration, the robot achieves its maximum velocity despite exhibiting the worst physical performance. Subsequently, the velocities in the simulator drops and gets stabilized for the other candidate points as the MFBO physical training progresses. Nevertheless, it consistently maintains a higher velocity than that observed in the physical realization. The cross-validation verifies the non-linear correlation of the addressed objective function across different data sources. Based on the simulation results, the Pearson's correlation coefficient is calculated with $K_{bi} = \begin{bmatrix} 1 & 0.601 \\ 0.601 & 1 \end{bmatrix}$. The high consistency between K_{bi} and K_f verifies that our estimated model has a high accuracy for domain adaptation, which leads to the realization of the MFBO with high efficiency.

8. Discussion

8.1. Summary of results

Section 7.1 lists several baseline algorithms, including AGS, RS, and OWD, where each method has performance limitations and known boundaries. AGS, while capable of refining the search space adaptively, becomes computationally expensive as the parameter space grows, particularly when a higher-resolution grid is needed for finer exploration. This approach requires an exponential number of evaluations as the dimensionality of the parameter space increases, leading to prohibitive computational costs for high-dimensional problems. Similarly, RS is generally less efficient because it lacks directed exploration, relying on random sampling of the parameter space without any strategy for selecting more promising areas to explore. In large or complex parameter spaces, RS can require a significant number of evaluations

to find an optimal or near-optimal solution, making it less suitable for applications like gait optimization where efficiency is key. Regarding the OWD baseline, previous studies demonstrated that it provides an ad hoc design for soft robot walking gait, but it lacks optimization or search processes. As a result, the walking patterns generated through OWD are limited in performance, especially in terms of walking speed and stability. Our paper aims to improve upon these limitations by using the MFBO framework to systematically explore and optimize the parameter space, leading to significantly enhanced walking speed and adaptability.

Based on the results in the above section, we synthesize the key findings from the experiments and highlight the overall contributions of our approach:

- **Optimization Performance:** The MFBO method, when combined with the CPG-based gait generation framework, demonstrates significant improvements in optimizing the soft robot's gait. Compared to baseline methods such as AGS, RS, and OWD, the MFBO approach consistently achieves faster convergence with fewer evaluations, highlighting its efficiency in balancing exploration and exploitation.
- **Computational Efficiency:** The introduction of task off-loading to the edge server plays a crucial role in improving computational efficiency. By off-loading complex computations, such as CPG parameter optimization and online training, to an external server, the system significantly reduces onboard processing demands, allowing for real-time adjustments to the robot's gait. This directly contributes to the robot's enhanced performance in both simulated and real-world environments.
- **Real-World Adaptability:** The MFBO approach effectively addresses the reality gap between simulation and physical experiments. By iteratively refining the gait parameters using real-world data, the soft robot adapts to environmental uncertainties, sensor noise, and hardware limitations. This adaptability is a key advantage over previous methods, which struggled with the sim-to-real transfer.
- **Gait Stability and Performance:** The optimized gait parameters not only improves the robot's walking speed but also enhances gait stability. The MFBO approach enables the robot to maintain a stable walking pattern without falling, even in real-world environments where surface irregularities and noise introduce challenges. This improvement is not achieved by baseline methods,

which either lack adaptability or require excessive computational resources to find near-optimal solutions.

In summary, the combination of MFBO and CPG-based gait generation offer significant advantages over traditional optimization methods. The improvements in computational efficiency, gait stability, and real-world adaptability showcase our approach as a highly effective solution for soft robotics applications, particularly in dynamic and uncertain environments.

8.2. Reality-gap analysis

Based on our observations in the experiments, our results indicate that the performance discrepancies between the simulated environment and the physical robot are primarily due to several key factors:

- **Environmental Conditions:** In simulation, environmental variables such as friction, surface texture, and ground unevenness are often simplified or modeled deterministically. However, in real-world environments, these factors can be highly variable and difficult to predict, leading to differences in the robot's interaction with its surroundings. For instance, the soft quadruped robot may experience varying friction levels or surface irregularities that are not fully captured in the simulation.
- **Sensor Noise:** The sensors used in the physical robot are subject to noise and inaccuracies that are not present in the idealized simulation environment. For example, sensors like IMUs and force sensors may provide imprecise data due to vibrations and time-drifting properties, which can lead to discrepancies between the expected and actual behavior of the robot.
- **Hardware Limitations:** The physical robot is constrained by hardware factors, such as motor performance, actuator limitations, and mechanical wear and tear, which are not always modeled accurately in the simulation. In particular, soft actuators exhibit nonlinearities and hysteresis properties, all of which can contribute to deviations from simulated performance.

These potential factors reveal the inherent challenges in transferring optimized control policies from simulation to real-world applications. The reality gap arises due to the complex interplay of environmental variability, sensor inaccuracies, and hardware limitations that are not fully captured by the simulation model. To mitigate this, the MFBO approach integrates both simulation and physical data, enabling continuous adaptation of the robot's control parameters to address discrepancies between the two environments, which allows the robot to compensate for the reality gap during the learning process gradually.

8.3. Answers to research questions

Based on the above results and analysis, the answers to RQs in Section 1 are given below:

- **RQ1:** The computation of the optimal gait pattern parameters is formulated as a stochastic black-box optimization problem, which is solved by the combination of Bayesian optimization and GP models. The GP model is effective for quantifying the uncertain relationship between gait parameters and walking quality and BO is computationally more efficient than other optimization techniques suitable for GP models.
- **RQ2:** The MFBO approach adapts the controller learned from the simulation models to the physical robots through online learning with the robots. The learning algorithm synergistically combines both simulation data and high-quality measurement data from robots. The sim-to-real adaption is so efficient that a small number of measurement data is necessary.
- **RQ3:** Physical experiments confirm that offloading to an edge server can achieve high-quality walking of the soft robot and meet all real-time constraints of the online learning algorithms.

8.4. Limitations and future research

Several limitations require further research. One of the limitations is the reliance on the fidelity of the simulation environment to approximate real-world dynamics. While the MFBO framework successfully integrates both simulation and experimental data to mitigate the reality gap, the approach remains sensitive to the accuracy of the simulation model. As a result, the performance of the physical robot may differ from simulation predictions, especially in unstructured environments. Another limitation is the dependence on reliable and timely communication between the robot and the edge server. In scenarios where network loss or latency occurs, the system lacks a graceful degradation strategy, which could lead to interruptions in the robot's operation. Graceful degradation refers to the ability of a system to maintain a reduced level of functionality when some of its components fail or when resources are constrained rather than failing [67]. Implementing a fail-safe mechanism that enables the robot to operate in a degraded mode with the onboard computation capability during periods of communication loss could significantly improve the system's robustness.

While this paper focuses on improving the gait performance of soft quadruped robots, the proposed framework is applicable to various types of soft robots and tasks, such as high-level navigation, manipulation, and locomotion in complex terrains. By addressing these challenges and enhancing the robustness and adaptability of the MFBO framework, this research has the potential to impact the deployment of soft robots in diverse industrial, medical, and exploratory applications, ensuring that they are more capable of operating autonomously in real-world settings.

9. Conclusion and future work

This study investigates advanced gait pattern design for a tendon-driven soft quadruped robot, with a specific focus on bridging the reality gap mitigation through learning-enhanced methods. The gait design exploits a parametric model based on Central Pattern Generators (CPG) with Hopf oscillators, whose parameters are determined through Bayesian Optimization (BO) using both simulation and experimental data. Furthermore, to ensure a smooth sim-to-real transfer, we refine the BO method and propose a multi-fidelity Bayesian Optimization (MFBO) approach for domain adaptation, thus efficiently compensating for modeling errors and overcoming the reality gap. Moreover, the entire training process utilizes an edge computing off-loading scheme with 5G communication, enhancing real-time data collection, training, control performance, and potential industrial applications. Future work includes investigation into walking properties for optimization and exploration of high-level navigation tasks on the soft quadruped robot platform. In addition, the communication dependence introduced by the off-loading scheme raises potential vulnerability due to communication loss. Hence, future work could extend this study by developing methods to enable the system to continue operation in a degraded mode and transition into a fail-safe state.

CRediT authorship contribution statement

Kaige Tan: Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Conceptualization. **Xuezhi Niu:** Software, Methodology, Investigation. **Qinglei Ji:** Software, Methodology, Investigation. **Lei Feng:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Martin Törnngren:** Writing – review & editing, Supervision, Project administration, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT in order to improve readability and language of the work. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research has been carried out as part of the TECoSA Vinnova Competence Center for Trustworthy Edge Computing Systems and Applications at KTH Royal Institute of Technology (www.tecosa.center.kth.se) and is also partly funded by the Entice project (Vinnova Advanced Digitalization program), Sweden. Lei Feng is also partly funded by KTH XPRES, Sweden. We acknowledge Gianfilippo Fornaro at KTH for his contribution to setting up the edge computing communication system.

Data availability

Data will be made available on request.

References

- [1] O. Yasa, Y. Toshimitsu, M.Y. Michelis, L.S. Jones, M. Filippi, T. Buchner, R.K. Katzschmann, An overview of soft robotics, *Annu. Rev. Control Robot. Auton. Syst.* 6 (2023) 1–29, <http://dx.doi.org/10.1146/annurev-control-062322-100607>.
- [2] R.S. Diteesawat, T. Helps, M. Taghavi, J. Rossiter, Electro-pneumatic pumps for soft robotics, *Sci. Robot.* 6 (51) (2021) eabc3721, <http://dx.doi.org/10.1126/scirobotics.abc3721>.
- [3] K. Tan, Q. Ji, L. Feng, M. Törnren, Shape estimation of a 3D printed soft sensor using multi-hypothesis extended Kalman filter, *IEEE Robot. Autom. Lett.* 7 (3) (2022) 8383–8390, <http://dx.doi.org/10.1109/LRA.2022.3187832>.
- [4] Z. Ma, D. Sameoto, A review of electrically driven soft actuators for soft robotics, *Micromachines* 13 (11) (2022) 1881, <http://dx.doi.org/10.3390/mi13111881>.
- [5] Q. Ji, S. Fu, K. Tan, S.T. Muralidharan, K. Lagrelius, D. Danelia, G. Andrikopoulos, X.V. Wang, L. Wang, L. Feng, Synthesizing the optimal gait of a quadruped robot with soft actuators using deep reinforcement learning, *Robot. Comput.-Integr. Manuf.* 78 (2022) 102382, <http://dx.doi.org/10.1016/j.rcim.2022.102382>.
- [6] N. Ebrahimi, C. Bi, D.J. Cappelleri, G. Ciuti, A.T. Conn, D. Faivre, N. Habibi, A. Hošovský, V. Iacovacci, I.S. Khalil, et al., Magnetic actuation methods in bio/soft robotics, *Adv. Funct. Mater.* 31 (11) (2021) 2005137, <http://dx.doi.org/10.1002/adfm.202005137>.
- [7] Y. Dong, L. Wang, N. Xia, Z. Yang, C. Zhang, C. Pan, D. Jin, J. Zhang, C. Majidi, L. Zhang, Untethered small-scale magnetic soft robot with programmable magnetization and integrated multifunctional modules, *Sci. Adv.* 8 (25) (2022) eabn8932, <http://dx.doi.org/10.1126/sciadv.abn8932>.
- [8] Q. Ji, S. Fu, L. Feng, G. Andrikopoulos, X.V. Wang, L. Wang, Omnidirectional walking of a quadruped robot enabled by compressible tendon-driven soft actuators, in: 2022 IEEE/RSJ International Conference on Intelligent Robots and Systems, IROS, IEEE, 2022, pp. 11015–11022, <http://dx.doi.org/10.1109/IROS47612.2022.9981314>.
- [9] D. Drotman, S. Jadhav, D. Sharp, C. Chan, M.T. Tolley, Electronics-free pneumatic circuits for controlling soft-legged robots, *Science Robotics* 6 (51) (2021) eaay2627, <http://dx.doi.org/10.1126/scirobotics.aay2627>.
- [10] K. Tan, Q. Ji, L. Feng, M. Törnren, Edge-enabled adaptive shape estimation of 3D printed soft actuators with Gaussian processes and unscented Kalman filters, *IEEE Trans. Ind. Electron.* 71 (3) (2024) 3044–3054, <http://dx.doi.org/10.1109/TIE.2023.3270505>.
- [11] P.M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, A. Del Prete, Optimization-based control for dynamic legged robots, *IEEE Trans. Robot.* 40 (2024) 43–63, <http://dx.doi.org/10.1109/TRO.2023.3324580>.
- [12] J.M. Bern, P. Banzet, R. Poranne, S. Coros, Trajectory optimization for cable-driven soft robot locomotion, in: *Robotics: Science and Systems*, 2019, pp. 1–8.
- [13] C. Della Santina, C. Duriez, D. Rus, Model-based control of soft robots: A survey of the state of the art and open challenges, *IEEE Control Syst. Mag.* 43 (3) (2023) 30–65, <http://dx.doi.org/10.1109/MCS.2023.3253419>.
- [14] H. Taheri, N. Mozayani, A study on quadruped mobile robots, *Mech. Mach. Theory* 190 (2023) 105448, <http://dx.doi.org/10.1016/j.mechmachtheory.2023.105448>.
- [15] G. Lan, M. De Carlo, F. van Diggelen, J.M. Tomczak, D.M. Roijers, A. Eiben, Learning directed locomotion in modular robots with evolvable morphologies, *Appl. Soft Comput.* (ISSN: 1568-4946) 111 (2021) 107688, <http://dx.doi.org/10.1016/j.asoc.2021.107688>.
- [16] M. Okulewicz, M. Zaborski, J. Mańdziuk, Self-adapting particle swarm optimization for continuous black box optimization, *Appl. Soft Comput.* 131 (2022) 109722, <http://dx.doi.org/10.1016/j.asoc.2022.109722>.
- [17] G. Nadizar, E. Medvet, S. Nichele, S. Pontes-Filho, An experimental comparison of evolved neural network models for controlling simulated modular soft robots, *Appl. Soft Comput.* 145 (2023) 110610, <http://dx.doi.org/10.1016/j.asoc.2023.110610>.
- [18] H. Jiang, T. Chen, J. Cao, J. Bi, G. Lu, G. Zhang, X. Rong, Y. Li, Stable skill improvement of quadruped robot based on privileged information and curriculum guidance, *Robot. Auton. Syst.* 170 (2023) 104550, <http://dx.doi.org/10.1016/j.robot.2023.104550>.
- [19] J. Ibarz, J. Tan, C. Finn, M. Kalakrishnan, P. Pastor, S. Levine, How to train your robot with deep reinforcement learning: lessons we have learned, *Int. J. Robot. Res.* 40 (4–5) (2021) 698–721, <http://dx.doi.org/10.1177/0278364920987859>.
- [20] M. Groshev, G. Baldoni, L. Cominardi, A. de la Oliva, R. Gazda, Edge robotics: Are we ready? An experimental evaluation of current vision and future directions, *Digit. Commun. Netw.* 9 (1) (2023) 166–174, <http://dx.doi.org/10.1016/j.dcan.2022.04.032>.
- [21] J. Yang, Y. Wang, Z. Li, Inverse order based optimization method for task offloading and resource allocation in mobile edge computing, *Appl. Soft Comput.* 116 (2022) 108361, <http://dx.doi.org/10.1016/j.asoc.2021.108361>.
- [22] K. Tan, L. Feng, G. Dán, M. Törnren, Decentralized convex optimization for joint task offloading and resource allocation of vehicular edge computing systems, *IEEE Trans. Veh. Technol.* 71 (12) (2022) 13226–13241.
- [23] T. Qiu, J. Chi, X. Zhou, Z. Ning, M. Atiquzzaman, D.O. Wu, Edge computing in industrial internet of things: Architecture, advances and challenges, *IEEE Commun. Surv. Tutor.* 22 (4) (2020) 2462–2488, <http://dx.doi.org/10.1109/COMST.2020.3009103>.
- [24] Y. Gao, W. Wei, X. Wang, D. Wang, Y. Li, Q. Yu, Trajectory tracking of multi-legged robot based on model predictive and sliding mode control, *Inform. Sci.* 606 (2022) 489–511, <http://dx.doi.org/10.1016/j.ins.2022.05.069>.
- [25] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, M. Hutter, Perceptive locomotion through nonlinear model-predictive control, *IEEE Trans. Robot.* 39 (5) (2023) 3402–3421, <http://dx.doi.org/10.1109/TRO.2023.3275384>.
- [26] Y. Shao, Y. Jin, X. Liu, W. He, H. Wang, W. Yang, Learning free gait transition for quadruped robots via phase-guided controller, *IEEE Robot. Autom. Lett.* 7 (2) (2021) 1230–1237, <http://dx.doi.org/10.1109/LRA.2021.3136645>.
- [27] Z. Song, J. Zhu, J. Xu, Gaits generation of quadruped locomotion for the CPG controller by the delay-coupled VDP oscillators, *Nonlinear Dynam.* 111 (19) (2023) 18461–18479, <http://dx.doi.org/10.1007/s11071-023-08783-2>.
- [28] H. Shi, B. Zhou, H. Zeng, F. Wang, Y. Dong, J. Li, K. Wang, H. Tian, M.Q.-H. Meng, Reinforcement learning with evolutionary trajectory generator: A general approach for quadrupedal locomotion, *IEEE Robot. Autom. Lett.* 7 (2) (2022) 3085–3092, <http://dx.doi.org/10.1109/LRA.2022.3145495>.
- [29] T. Miki, J. Lee, J. Hwangbo, L. Wellhausen, V. Koltun, M. Hutter, Learning robust perceptive locomotion for quadrupedal robots in the wild, *Science Robotics* 7 (62) (2022) eabk2822, <http://dx.doi.org/10.1126/scirobotics.abk2822>.
- [30] J. Kim, D.X. Ba, H. Yeom, J. Bae, Gait optimization of a quadruped robot using evolutionary computation, *J. Bionic Eng.* 18 (2021) 306–318.
- [31] D. Widmer, D. Kang, B. Sukhija, J. Hübotter, A. Krause, S. Coros, Tuning legged locomotion controllers via safe bayesian optimization, in: *Conference on Robot Learning*, PMLR, 2023, pp. 2444–2464.
- [32] T. Srinivas, A.K.K. Madhusudhan, L. Manohar, N.M. Stephen Pushpagiri, K.C. Ramanathan, M. Janardhanan, I. Nielsen, Valkyrie—Design and development of gaits for quadruped robot using particle swarm optimization, *Appl. Sci.* 11 (16) (2021) 7458.
- [33] K. Zhou, Z. Liu, Y. Qiao, T. Xiang, C.C. Loy, Domain generalization: A survey, *IEEE Trans. Pattern Anal. Mach. Intell.* 45 (4) (2022) 4396–4415.
- [34] D. Horváth, G. Erdős, Z. Istene, T. Horváth, S. Földi, Object detection using sim2real domain randomization for robotic applications, *IEEE Trans. Robot.* 39 (2) (2022) 1225–1243.
- [35] F. Muratore, C. Eilers, M. Gienger, J. Peters, Data-efficient domain randomization with bayesian optimization, *IEEE Robot. Autom. Lett.* 6 (2) (2021) 911–918.
- [36] N. Jaquier, M.C. Welle, A. Gams, K. Yao, B. Fichera, A. Billard, A. Ude, T. Asfour, D. Kragic, Transfer learning in robotics: An upcoming breakthrough? A review of promises and challenges, *Int. J. Robot. Res.* (2023) 02783649241273565.

- [37] X. Zhang, C. Wang, L. Sun, Z. Wu, X. Zhu, M. Tomizuka, Efficient sim-to-real transfer of contact-rich manipulation skills with online admittance residual learning, in: *Conference on Robot Learning*, PMLR, 2023, pp. 1621–1639.
- [38] P. Perdikaris, G.E. Karniadakis, Model inversion via multi-fidelity Bayesian optimization: a new paradigm for parameter estimation in haemodynamics, and beyond, *J. R. Soc. Interface* 13 (118) (2016) 20151107, <http://dx.doi.org/10.1098/rsif.2015.1107>.
- [39] A. Marco, F. Berkenkamp, P. Hennig, A.P. Schoellig, A. Krause, S. Schaal, S. Trimpe, Virtual vs. real: Trading off simulations and physical experiments in reinforcement learning with Bayesian optimization, in: *2017 IEEE International Conference on Robotics and Automation, ICRA, IEEE*, 2017, pp. 1557–1563, <http://dx.doi.org/10.1109/ICRA.2017.7989186>.
- [40] T. Wei, S. Wang, J. Zhong, D. Liu, J. Zhang, A review on evolutionary multitask optimization: Trends and challenges, *IEEE Trans. Evol. Comput.* 26 (5) (2022) 941–960, <http://dx.doi.org/10.1109/TEVC.2021.3139437>.
- [41] Y. Zhang, Q. Yang, A survey on multi-task learning, *IEEE Trans. Knowl. Data Eng.* 34 (12) (2022) 5586–5609, <http://dx.doi.org/10.1109/TKDE.2021.3070203>.
- [42] S. Horváth, A. Klein, P. Richtarik, C. Archambeau, Hyperparameter transfer learning with adaptive complexity, in: A. Banerjee, K. Fukumizu (Eds.), *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, in: *Proceedings of Machine Learning Research*, vol. 130, PMLR, 2021, pp. 1378–1386.
- [43] H. Wang, Y. Jin, C. Yang, L. Jiao, Transfer stacking from low-to high-fidelity: A surrogate-assisted bi-fidelity evolutionary algorithm, *Appl. Soft Comput.* 92 (2020) 106276, <http://dx.doi.org/10.1016/j.asoc.2020.106276>.
- [44] E. Lau, V. Srivastava, S.D. Bopardikar, A multi-fidelity Bayesian approach to safe controller design, *IEEE Control Syst. Lett.* 7 (2023) 2904–2909, <http://dx.doi.org/10.1109/LCSYS.2023.3290475>.
- [45] M.G. Fernández-Godino, Review of multi-fidelity models, *Adv. Comput. Sci. Eng.* 1 (4) (2023) 351–400, <http://dx.doi.org/10.3934/acse.2023015>.
- [46] P. Huang, L. Zeng, X. Chen, K. Luo, Z. Zhou, S. Yu, Edge robotics: Edge-computing-accelerated multirobot simultaneous localization and mapping, *IEEE Internet Things J.* 9 (15) (2022) 14087–14102.
- [47] R.A. de Omena, D.F. Santos, A. Perkusich, D.C. Valadares, Two-tier MPC architecture for AGVs navigation assisted by edge computing in an industrial scenario, *Internet Things* 21 (2023) 100666.
- [48] A.K. Tanwani, N. Mor, J. Kubiawicz, J.E. Gonzalez, K. Goldberg, A fog robotics approach to deep robot learning: Application to object recognition and grasp planning in surface decluttering, in: *2019 International Conference on Robotics and Automation, ICRA, IEEE*, 2019, pp. 4559–4566.
- [49] Y. Shi, W. Shen, L. Wang, F. Longo, L. Nicoletti, A. Padovano, A cognitive digital twins framework for human-robot collaboration, *Procedia Comput. Sci.* 200 (2022) 1867–1874.
- [50] K. Tan, L. Feng, M. Törngren, Collaborative collision avoidance of connected vehicles using ADMM with PI-regulated Lagrangian multipliers, in: *2023 IEEE 19th International Conference on Automation Science and Engineering, CASE, IEEE*, 2023, pp. 1–8.
- [51] S. Han, H. Ma, A. Taherkordi, D. Lan, Y. Chen, Privacy-preserving data integration scheme in industrial robot system based on fog computing and edge computing, *IET Commun.* 18 (7) (2024) 461–476.
- [52] H. Zhu, Q. Chen, X. Zhu, W. Yao, X. Chen, Edge computing powers aerial swarms in sensing, communication, and planning, *The Innovation* 4 (6) (2023).
- [53] W. Xu, Z. Yang, D.W.K. Ng, M. Levorato, Y.C. Eldar, M. Debbah, Edge learning for B5G networks with distributed signal processing: Semantic communication, edge computing, and wireless sensing, *IEEE J. Sel. Top. Signal Process.* 17 (1) (2023) 9–39.
- [54] L. Bliet, A. Guijt, R. Karlsson, S. Verwer, M. de Weerd, Benchmarking surrogate-based optimisation algorithms on expensive black-box functions, *Appl. Soft Comput.* 147 (2023) 110744, <http://dx.doi.org/10.1016/j.asoc.2023.110744>.
- [55] J. Snoek, H. Larochelle, R.P. Adams, Practical Bayesian optimization of machine learning algorithms, in: *Advances in Neural Information Processing Systems*, vol. 25, Curran Associates, Inc., 2012, pp. 1–9.
- [56] C. Zhou, K. Low, Design and locomotion control of a biomimetic underwater vehicle with fin propulsion, *IEEE/ASME Trans. Mechatronics* 17 (1) (2011) 25–35, <http://dx.doi.org/10.1109/TMECH.2011.2175004>.
- [57] M.A. Álvarez, L. Rosasco, N.D. Lawrence, Kernels for vector-valued functions: A review, *Found. Trends Mach. Learn.* (ISSN: 1935-8237) 4 (3) (2012) 195–266, <http://dx.doi.org/10.1561/22000000036>.
- [58] Y. Shi, G. Zhao, M. Wang, Y. Xu, An adaptive grid search algorithm for fitting spherical target of terrestrial LiDAR, *Measurement* 198 (2022) 111430, <http://dx.doi.org/10.1016/j.measurement.2022.111430>.
- [59] D. Delahaye, S. Chaimatanan, M. Mongeau, Simulated annealing: From basics to applications, in: *Handbook of metaheuristics*, Springer, 2019, pp. 1–35, http://dx.doi.org/10.1007/978-3-319-91086-4_1.
- [60] S. Andradóttir, An overview of simulation optimization via random search, in: *Handbooks in Operations Research and Management Science*, vol. 13, Elsevier, 2006, pp. 617–631, [http://dx.doi.org/10.1016/S0927-0507\(06\)13020-0](http://dx.doi.org/10.1016/S0927-0507(06)13020-0).
- [61] L. Pronzato, W.G. Müller, Design of computer experiments: space filling and beyond, *Stat. Comput.* 22 (2012) 681–701.
- [62] F. Hutter, H.H. Hoos, K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, in: *Learning and Intelligent Optimization: 5th International Conference, LION 5, Rome, Italy, January 17–21, 2011. Selected Papers* 5, Springer, 2011, pp. 507–523, http://dx.doi.org/10.1007/978-3-642-25566-3_40.
- [63] L. Bliet, H.R. Verstraete, M. Verhaegen, S. Wahls, Online optimization with costly and noisy measurements using random Fourier expansions, *IEEE Trans. Neural Netw. Learn. Syst.* 29 (1) (2016) 167–182, <http://dx.doi.org/10.1109/TNNLS.2016.2615134>.
- [64] P. Saves, R. Lafage, N. Bartoli, Y. Diouane, J. Bussemaker, T. Lefebvre, J.T. Hwang, J. Morlier, J.R. Martins, SMT 2.0: A surrogate modeling toolbox with a focus on hierarchical and mixed variables Gaussian processes, *Adv. Eng. Softw.* 188 (2024) 103571, <http://dx.doi.org/10.1016/j.advengsoft.2023.103571>.
- [65] L. Bliet, S. Verwer, M. de Weerd, Black-box combinatorial optimization using models with integer-valued minima, *Ann. Math. Artif. Intell.* 89 (7) (2021) 639–653, <http://dx.doi.org/10.1007/s10472-020-09712-4>.
- [66] R. Dürichen, M.A. Pimentel, L. Clifton, A. Schweikard, D.A. Clifton, Multitask Gaussian processes for multivariate physiological time-series analysis, *IEEE Trans. Biomed. Eng.* 62 (1) (2014) 314–322, <http://dx.doi.org/10.1109/TBME.2014.2351376>.
- [67] P. Weiss, S. Steinhorst, Predictable timing behavior of gracefully degrading automotive systems, *Des. Autom. Embedded Syst.* 27 (1) (2023) 103–138.